

华南理工大学硕士学位论文

# LaTeX 模板使用说明

作者姓名

指导教师：xxx 教授

华南理工大学

2025 年 1 月 6 日

# 目 录

|                                     |     |
|-------------------------------------|-----|
| 主要符号对照表 .....                       | III |
| 第一章 涵道风扇式无人机的姿态控制 .....             | 1   |
| 1.1 增量非线性动态逆控制与控制分配理论 .....         | 1   |
| 1.1.1 增量非线性动态逆 .....                | 2   |
| 1.1.2 控制分配理论 .....                  | 3   |
| 1.2 姿态动力学模型简化 .....                 | 4   |
| 1.3 基于 INDI 和优先级控制分配的姿态控制方案 .....   | 6   |
| 1.3.1 INDI 姿态控制架构 .....             | 7   |
| 1.3.2 陀螺力矩补偿 .....                  | 13  |
| 1.3.3 优先级控制分配 .....                 | 14  |
| 1.4 实验和结果分析 .....                   | 17  |
| 1.4.1 仿真实验 .....                    | 19  |
| 1.4.2 实际飞行实验 .....                  | 22  |
| 1.5 本章小结 .....                      | 24  |
| 第二章 涵道风扇式无人机的位置控制 .....             | 25  |
| 2.1 位置动力学模型简化 .....                 | 26  |
| 2.2 基于 INDI 的位置控制器设计 .....          | 27  |
| 2.2.1 INDI 位置控制架构 .....             | 28  |
| 2.2.2 期望姿态角与拉力解算 .....              | 31  |
| 2.3 实验结果和分析 .....                   | 34  |
| 2.3.1 仿真实验 .....                    | 34  |
| 2.3.2 实际飞行实验 .....                  | 37  |
| 2.4 本章小结 .....                      | 42  |
| 第三章 涵道风扇式无人机的轨迹规划设计 .....           | 43  |
| 3.1 RRT 算法及其改进算法理论介绍 .....          | 43  |
| 3.1.1 RRT 算法 .....                  | 43  |
| 3.1.2 RRT-Connect 算法和 RRT* 算法 ..... | 45  |

|                                  |    |
|----------------------------------|----|
| 3.2 C-RRT* 算法 .....              | 48 |
| 3.2.1 冗余节点删除策略 .....             | 49 |
| 3.2.2 C-RRT* 算法流程 .....          | 50 |
| 3.3 基于 B 样条曲线的轨迹优化 .....         | 51 |
| 3.3.1 B 样条曲线理论 .....             | 52 |
| 3.4 仿真实验分析 .....                 | 53 |
| 3.4.1 C-RRT* 算法综合性能验证与对比分析 ..... | 53 |
| 3.5 本章小结 .....                   | 54 |
| 参考文献 .....                       | 55 |

## 主要符号对照表

$O_e - X_e Y_e Z_e$ -地面坐标系

$P^e = [x^e \ y^e \ z^e]^T$ -地面系下的位置

$V^b = [u \ v \ w]^T$ -机体系下的速度

$\theta$ -俯仰角

$\omega^b = [p \ q \ r]^T$ -机体系下的角速度

$F^b$ -机体系下受到的除重力外的合力

$g$ -当地的重力加速度

$J^b = \text{diag}[J_x \ J_y \ J_z]$ -机体系下的转动惯量

$\sigma_d$ -涵道扩压比

$V_e$ -涵道出口风速

$T_p$ -风扇升力

$\Omega$ -风扇转速

$k_{fan}$ -风扇拉力常系数

$C_{D,x}$ 、 $C_{D,y}$ 、 $C_{D,z}$ -沿机体轴的阻力系数

$l_a$ -机身空气动力中心与机身重心的距离

$\gamma$ -环绕涵道角度变量

$V_z$ -气流相对于机体的轴向速度

$\alpha_d$ -迎角

$C_{d,d}$  涵道翼型阻力曲线

$C_{duct}$  - 常值比例系数

$k_\delta$ -控制舵面升力系数

$J_{fan}$ -涵道风扇转动惯量

$\varphi_0$ -固定气动面安装角

$O_b - X_b Y_b Z_b$ -机体系坐标系

$V^e = [v_x^e \ v_y^e \ v_z^e]^T$ -地面系下的速度

$\psi$ -偏航角

$\varphi$ -滚转角

$R_e^b$ -机体系到地面系的旋转矩阵

$m$ -无人机总质量

$M^b$ -机体系下受到的合外力矩

$S$ -桨盘面积

$\rho$ -空气密度

$V'$ -涵道风扇诱导速度

$T_l$ -涵道体升力

$R$ -风扇半径

$k_q$ -风扇扭矩常系数

$S_x$ 、 $S_y$ 、 $S_z$ -沿机体轴的截面面积

$V_a^b = [u_r \ v_r \ w_r]^T$ -空速

$V_r$ -气流相对于机体的径向速度

$q_d$ -涵道周围的动态压力

$C_{l,d}$ -涵道翼型升力曲线

$c_d$ -涵道翼型弦长

$l_d$ -重心与涵道空气动力中心之间的距离

$\delta_i$ -控制舵面偏转角

$d_{af}$ 、 $d_{ds}$ -风扇扭矩常系数

$W^e$ -环境风速



## 第一章 涵道风扇式无人机的姿态控制

基于第二章建立的 DFUAV 非线性系统动态方程分析，其多源力与力矩耦合叠加效应导致姿态动态模型尤为复杂。如滚转角的变化会在俯仰通道和偏航通道上都产生或者间接产生力矩影响，并且三维力矩都与环境风速呈现强相关性。最常用的姿态控制方法基于反馈线性化，比如不考虑系统的数学模型的 PID 控制算法，虽然该方法面对非线性系统有一定的鲁棒性，但仅通过比例-积分-微分的线性组合应对内外因素产生的力矩影响难免会使系统状态出现超调和滞后等问题。此外，可以使用基于系统模型的方法考虑内外因素对系统状态的影响，但在系统模型参数过多并且不精确的情况下也将为控制算法的设计和系统调试带来很大工作量。基于上述分析，姿态控制算法设计的核心在于如何对难以测得的力矩进行在线补偿。

本章安排如下：首先简要概述增量非线性动态逆控制和控制分配的理论以及二者的结合，然后对 DFUAV 的姿态动力学模型进行简化处理，方便下文姿态控制算法的设计。接下来给出了基于 INDI 和优先级控制分配的 DFUAV 的姿态控制方案设计，最后为验证方案有效性，最后进行了仿真实验和飞行实验验证。

### 1.1 增量非线性动态逆控制与控制分配理论

NDI(非线性动态逆)是一种基于模型的非线性控制方法，其核心在于在掌握系统模型的情况下，将非线性系统在工作点处进行反馈线性化，然后通过求逆运算来消除系统中的非线性项。当系统模型参数不精确或者对系统机理认识模糊的情况下，NDI 方法的控制效果将表现不佳。而 INDI(增量式非线性动态逆)相比于 NDI 采用了增量式的控制策略，仅要求对系统的输入通道建模。INDI 在未能充分掌握系统模型的情况下，通过增量式的求逆策略来实时补偿未知扰动与建模误差，得到线性化的系统动力学，进而可以通过常规的线性系统的控制方法来控制<sup>[1]</sup>。因此 INDI 方法对系统模型的依赖程度较小，是一种基于传感器的方法。此外，INDI 方法在设计控制律时主要关注系统的输入输出特性，无需考虑应如何将控制律计算出的控制量分配到各个执行机构上。而后者所描述的任务将由控制分配算法来实现，实现过程中也无需考虑系统内部的复杂非线性因素与动态干扰。因此，采用 INDI 控制方法有助于将姿态控制算法设计与控制分配算法分离开，实现分层控制。

本节首先将从一般性的系统推导 INDI 是如何实现增量控制的，然后介绍控制分配理论以及二者的结合。

### 1.1.1 增量非线性动态逆

考虑如下普通的非线性动力系统：

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}) + \mathbf{g}(\mathbf{x}, \mathbf{u}) \quad (1-1)$$

$$\mathbf{y} = \mathbf{h}(\mathbf{x}) \quad (1-2)$$

上式中  $\mathbf{x} \in \mathbb{R}^n$  为  $n$  维系统状态变量， $\mathbf{u} \in \mathbf{U} \in \mathbb{R}^p$  为  $p$  维系统输入变量，其中  $\mathbf{U}$  是容许控制集合，表示对输入变量的约束， $\mathbf{y} \in \mathbb{R}^m$  为  $m$  维系统输出变量。映射关系  $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^n$ ， $\mathbf{g} : \mathbb{R}^n \times \mathbb{R}^p \rightarrow \mathbb{R}^n$ ， $\mathbf{h} : \mathbb{R}^n \rightarrow \mathbb{R}^m$ 。

式(1-1)中系统状态的一阶泰勒展开式为：

$$\dot{\mathbf{x}} = \dot{\mathbf{x}}_0 + \mathbf{f}'(\mathbf{x})(\mathbf{x} - \mathbf{x}_0) + \left. \frac{\partial \mathbf{g}}{\partial \mathbf{x}} \right|_{\mathbf{x}=\mathbf{x}_0} (\mathbf{x} - \mathbf{x}_0) + \left. \frac{\partial \mathbf{g}}{\partial \mathbf{u}} \right|_{\mathbf{u}=\mathbf{u}_0} (\mathbf{u} - \mathbf{u}_0) \quad (1-3)$$

然后对式(1-2)中的输出表达式求一次导数，结合复合函数求导的链式法则与式(1-3)，可以得到动力系统输入与输出之间的关系：

$$\dot{\mathbf{y}} = \underbrace{\left. \frac{\partial \mathbf{h}}{\partial \mathbf{x}} \right|_{\mathbf{x}=\mathbf{x}_0} \dot{\mathbf{x}}_0}_{\dot{\mathbf{y}}_0} + \left. \frac{\partial \mathbf{h}}{\partial \mathbf{x}} \right|_{\mathbf{x}=\mathbf{x}_0} \left[ \mathbf{f}'(\mathbf{x})(\mathbf{x} - \mathbf{x}_0) + \left. \frac{\partial \mathbf{g}}{\partial \mathbf{x}} \right|_{\mathbf{x}=\mathbf{x}_0} (\mathbf{x} - \mathbf{x}_0) + \left. \frac{\partial \mathbf{g}}{\partial \mathbf{u}} \right|_{\mathbf{u}=\mathbf{u}_0} (\mathbf{u} - \mathbf{u}_0) \right] \quad (1-4)$$

INDI 理论的一个核心假设是时间尺度分离法则<sup>[2]</sup>，该法则基于系统动力学的时间尺度特性，指出系统内部状态变量的动态响应速率要明显慢于外部输入信号的时变特性。基于这一动力学特性差异，相比于外部输入信号的导数项，系统状态变量的导数项可以视为高阶小量并忽略不计，从而有效地简化系统的动态方程。

基于上述核心假设，可以将式(1-4)中系统状态的导数项  $(\mathbf{x} - \mathbf{x}_0)$  忽略，得到：

$$\begin{aligned} \dot{\mathbf{y}} &= \dot{\mathbf{y}}_0 + \left. \frac{\partial \mathbf{h}}{\partial \mathbf{x}} \right|_{\mathbf{x}=\mathbf{x}_0} \left. \frac{\partial \mathbf{g}}{\partial \mathbf{u}} \right|_{\mathbf{u}=\mathbf{u}_0} (\mathbf{u} - \mathbf{u}_0) \\ &= \dot{\mathbf{y}}_0 + \mathbf{G}(\mathbf{u} - \mathbf{u}_0) \end{aligned} \quad (1-5)$$

其中  $\dot{\mathbf{y}}_0$  为系统当前时刻输出的导数， $\mathbf{G} \in \mathbb{R}^{m \times p}$ ， $\mathbf{u}_0$  为系统当前时刻的控制输入。使用下标  $(\cdot)_d$  表示该变量的期望值，那么根据公式(1-5)，可以得到期望的输入变量  $\mathbf{u}_d$  表示为：

$$\begin{aligned} \mathbf{u}_d &= \mathbf{u}_0 + \Delta \mathbf{u} \\ &= \mathbf{u}_0 + \mathbf{G}^{-1}(\dot{\mathbf{y}}_d - \dot{\mathbf{y}}) \end{aligned} \quad (1-6)$$

新的控制输入是在当前时刻的控制输入基础上加上一个控制增量  $\Delta u$  得到。通过对控制效率矩阵求逆，并且找到期望的输出导数与当前时刻的输出导数的误差，可以得到控制增量。

### 1.1.2 控制分配理论

继续考虑 3.1.1 小节中引入的非线性动力系统。对于一个过驱动系统，其系统输入的维度大于系统输出的维度，即  $p > m$ ，表明执行机构存在冗余。将  $g(x, u)$  做矩阵分解得到：

$$g(x, u) = DK(x, u) \quad (1-7)$$

其中  $D \in \mathbb{R}^{n \times m}$ ，映射关系  $K : \mathbb{R}^n \times \mathbb{R}^p \rightarrow \mathbb{R}^m$ 。

引入  $m$  维虚拟控制输入  $\nu$ ：

$$\nu = K(x, u) \quad (1-8)$$

由于实际控制输入的约束  $U$  存在，将实际控制输入变换为虚拟控制输入后有  $\nu \in \Phi \in \mathbb{R}^m$ 。容许控制集合  $U$  通过映射  $K$  从  $\mathbb{R}^p$  空间映射到  $\mathbb{R}^m$  空间得到可达控制集合  $\Phi$ ，如果  $\nu \in \Phi$ ，则称虚拟控制输入为可达，否则为不可达。虚拟控制输入的物理意义一般为力或力矩。

对  $\nu$  做一阶泰勒展开并且同样忽略关于系统状态的导数项，得到：

$$\begin{aligned} \nu &= \nu_0 + \left. \frac{\partial K}{\partial u} \right|_{u=u_0} (u - u_0) \\ &= \nu_0 + B(u - u_0) \end{aligned} \quad (1-9)$$

其中  $\nu_0$  表示系统当前时刻的虚拟控制输入， $B \in \mathbb{R}^{m \times p}$ 。将  $(u - u_0)$  代入公式(1-5)，得到：

$$\begin{aligned} \dot{y} &= \dot{y}_0 + GB^\dagger(\nu - \nu_0) \\ \Rightarrow \dot{y}_d - \dot{y}_0 &= GB^\dagger \Delta \nu \end{aligned} \quad (1-10)$$

其中  $B^\dagger$  表示矩阵  $B$  的伪逆矩阵，有以下关系：

$$u = B^\dagger \nu \quad (1-11)$$

$$B^\dagger = B^T (BB^T)^{-1} \quad (1-12)$$

公式(1-11)中直接表示虚拟控制输入  $\nu$  与实际控制输入  $u$  之间映射关系的方法就是伪逆法。在分层控制架构中，系统采用上层决策-底层执行的层级化设计模式：上层控制算法承担决策功能，专注于在线求解系统动力学方程生成期望控制指令，避免了执行机构物理约束对优化问题的干扰，降低了计算复杂度；底层控制分配算法则负责执行任务，采用优化计算方法将上层指令解析为各执行机构的物理控制量，实现控制指令在冗余执行机构中的最优分配。文献[3] 中讨论了层级化设计方法和全阶最优控制设计的联系。采用独立的控制分配算法，优势之一是可以考虑执行机构的物理约束，比如当某个执行机构饱和时，在条件允许的情况下其余执行机构可以用于弥补因该执行机构饱和而损失的控制效能。图1-1展示了基于 INDI 的上层控制算法和基于伪逆法的底层控制分配算法的控制系统框图。

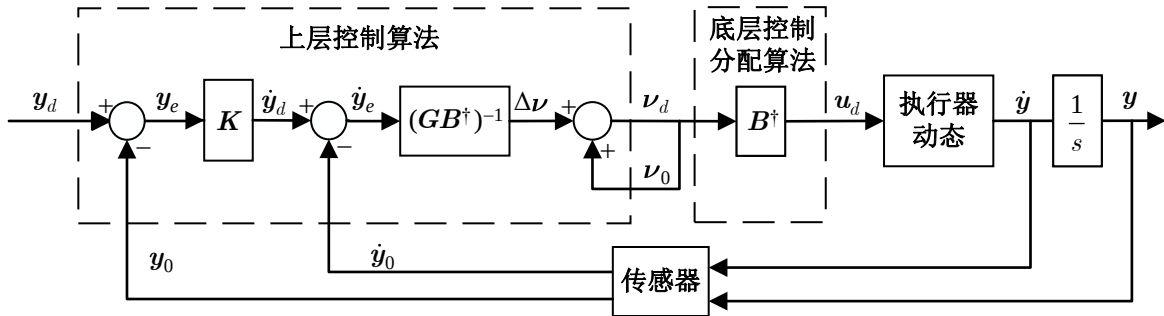


图 1-1 基于 INDI 和伪逆法的控制系统框图

需特别指出的是，尽管可以设计出渐进稳定的控制算法，但是受矩阵  $B$  奇异特性及执行机构物理约束的影响，所推导的期望虚拟控制输入  $\nu_d$  可能属于不可达集。这一现象在本质上源于伪逆法在控制分配问题中的固有局限性：当控制指令超出执行机构可达包络时，将导致控制分配问题退化为无可行解的数学约束条件，因此下文在控制分配算法的设计中将参考文献[4]，采用优先级控制分配方法。该方法旨在通过优先级排序的方式，优先满足高优先级的控制输入分量以达到更大范围的容许控制。

## 1.2 姿态动力学模型简化

根据第二章的建模总结可知，DFUAV 的姿态动力学模型涉及到多个系统状态变量间的耦合并且与许多模型参数和环境风速有关，尤其是角速度动态方程(??)中涉及到的多源力矩模型。为便于下文姿态控制算法的设计，本节将对所总结的力矩模型进行合理且适当的简化，根据现有条件区分可控力矩与不可控力矩，然后分别处理。

根据式??可知，DFUAV 在机体坐标系下的合外力矩可以分解为：

$$\mathbf{M}^b = \mathbf{M}_{fan}^b + \mathbf{M}_{aero}^b + \mathbf{M}_{duct}^b + \mathbf{M}_{vane}^b + \mathbf{M}_{gyro}^b + \mathbf{M}_{flap}^b \quad (1-13)$$

由于涵道风扇的转速  $\Omega$  可由电调读取，机体的角速度  $\boldsymbol{\omega}^b$  可由机载的 IMU 测量得到，结合表??中的模型参数，可以计算出涵道风扇的扭矩  $\mathbf{M}_{fan}^b$  和陀螺力矩  $\mathbf{M}_{gyro}^b$ 。

模型假设气流相对于机体的速度在  $\mathbf{O}_b - \mathbf{Z}_b$  轴的分量为零，即  $V_0 = 0$ 。根据式??以及式??，可将涵道出口风速  $V_e$  简化为：

$$V_e = \sqrt{\frac{k_{fan}}{\sigma_d \rho S}} \Omega = k_f \Omega \quad (1-14)$$

在这种假设下，涵道出口风速  $V_e$  与风扇转速  $\Omega$  之间呈正比关系。不妨设  $V_e = k_f \Omega$ ，其中  $k_f = \sqrt{\frac{k_{fan}}{\sigma_d \rho S}}$  可以根据表??中的参数计算得到。然后根据控制舵面的偏转角  $\delta_i$  和涵道力臂的长度，可计算出控制舵面产生的力矩  $\mathbf{M}_{vane}^b$ 。

对于在固定气动面上产生的反扭矩  $\mathbf{M}_{flap}^b$ ，由于  $V_e$  与  $\Omega$  为正比关系，可将式??简化为：

$$\mathbf{M}_{flap}^b = \begin{bmatrix} 0 \\ 0 \\ k_{flap} \Omega^2 \end{bmatrix} \quad (1-15)$$

现在考虑风扇扭矩  $\mathbf{M}_{fan}^b$ 、固定气动面上的反扭矩  $\mathbf{M}_{flap}^b$  和控制舵面在  $\mathbf{O}_b - \mathbf{Z}_b$  轴的偏航力矩。根据第二章的力矩分析，仅有上述三部分分量会对机体的偏航力矩产生影响。为使机体的偏航角保持稳定，有如下关系：

$$\begin{aligned} -k_q \Omega^2 + k_{flap} \Omega^2 + k_\delta k_f^2 l_2 (\delta_1 + \delta_2 + \delta_3 + \delta_4) \Omega^2 &= 0 \\ \Rightarrow -k_q + k_{flap} + k_\delta k_f^2 l_2 (\delta_1 + \delta_2 + \delta_3 + \delta_4) &= 0 \end{aligned} \quad (1-16)$$

式1-15中等式约去风扇转速变量  $\Omega$  后，只有控制舵面的偏转角是变量，其他均为确定的模型参数。这表明了当机体偏航角稳定时，所需的控制舵面偏转角是一定的。由于  $k_{flap}$  与固定气动面的设计形状有关，不失一般性，本文假设固定气动面在恰当的设计下可以使得  $k_{flap} = k_q$ ，即：

$$\mathbf{M}_{flap}^b + \mathbf{M}_{fan}^b = \mathbf{0} \quad (1-17)$$

对于在机身上产生的气动力矩  $\mathbf{M}_{aero}^b$  和由于涵道翼型产生的力矩  $\mathbf{M}_{duct}^b$ ，由于机体

相对于气流的速度  $\mathbf{V}_a^b$  在现有条件下是未知的，所以将其视为不可控力矩来作为未建模动态力矩。使用  $\mathbf{M}_a^b$  统一表示：

$$\mathbf{M}_a^b = \mathbf{M}_{aero}^b + \mathbf{M}_{duct}^b \quad (1-18)$$

$\mathbf{M}_a^b$  可以描述为地面坐标系下的速度  $\mathbf{V}^e$ 、姿态  $\boldsymbol{\eta}$  和环境风速  $\mathbf{W}^e$  的函数，即：

$$\mathbf{M}_a^b = \mathbf{M}_a^b(\mathbf{V}^e, \boldsymbol{\eta}, \mathbf{W}^e) \quad (1-19)$$

综合以上分析，简化后的气动力矩模型可以写为：

$$\mathbf{M}^b = \mathbf{M}_{vane}^b + \mathbf{M}_{gyro}^b + \mathbf{M}_a^b \quad (1-20)$$

与力矩模型相关的角速度动态方程对应的可以简写为：

$$\begin{aligned} \dot{p} &= \frac{1}{J_x} \left\{ (J_y - J_z)qr + [-l_1 k_\delta V_e^2 (\delta_1 - \delta_3) - J_{fan} \Omega q + \mathbf{M}_{ax}^b] \right\} \\ \dot{q} &= \frac{1}{J_y} \left\{ (J_z - J_x)pr + [l_1 k_\delta V_e^2 (\delta_4 - \delta_2) + J_{fan} \Omega p + \mathbf{M}_{ay}^b] \right\} \\ \dot{r} &= \frac{1}{J_z} \left\{ (J_x - J_y)pq + [l_2 k_\delta V_e^2 (\delta_1 + \delta_2 + \delta_3 + \delta_4) + \mathbf{M}_{az}^b] \right\} \end{aligned} \quad (1-21)$$

上述简化的力矩模型将可控力矩与不可控力矩分开。控制舵面合力矩  $\mathbf{M}_{vane}^b$  作为可控力矩由下文设计的姿态控制算法来计算期望力矩 (即期望的控制舵面偏转角度)。陀螺力矩  $\mathbf{M}_{gyro}^b$  同样作为可控力矩，但该力矩会对 DFUAV 的姿态造成负面影响，下文将根据其产生的机理设计对应的陀螺力矩补偿算法加以补偿。而  $\mathbf{M}_a^b$  作为不可控力矩将通过姿态控制算法在线补偿。

### 1.3 基于 INDI 和优先级控制分配的姿态控制方案

根据本章开篇时的分析，姿态控制算法设计的核心在于如何对难以测得的力矩进行在线补偿。在第二节姿态动力学模型简化中，将难以测得的力矩视为了未建模动态力矩  $\mathbf{M}_a^b$ 。基于 INDI 的控制架构设计，其优势主要体现在模型的鲁棒性与未建模误差的动态补偿能力。式(1-21)中的形式已十分适用于 INDI 姿态控制器的设计：一方面，姿态子系统的输入通道的模型机理已经明确，由四片独立的控制舵面产生偏转角作为输入信号；另一方面，由于未建模动态力矩  $\mathbf{M}_a^b$  作用于角加速度，所以可用机载 IMU 高频测量角速度并估算角加速度，并根据 INDI 的增量式求逆策略在线补偿未建模动态力矩

$\mathbf{M}_a^b$ 。在所构建的控制架构中，控制输入分为两部分：INDI 输入与反馈输入。INDI 输入针对  $\mathbf{M}_a^b$  进行补偿，将非线性系统转化为线性形式；而反馈输入针对此线性形式设计反馈控制律，保证系统的稳定性和鲁棒性。

根据优先级控制分配的思想，需要将控制输入分量进行优先级排序。由于 INDI 输入包含对未建模动态力矩的补偿，反馈输入使用线性的状态反馈设计，与系统响应相关。系统能够稳定运行的前提是首先消除未建模动态力矩的影响，将非线性系统化为线性系统，然后再设计反馈控制律，确保系统的稳定性与响应能力。因此本文中将 INDI 输入视为高优先级分量，反馈输入视为低优先级分量。确保 INDI 输入不会产生分配误差，在控制舵面受到物理约束的情况下仅在反馈输入中产生分配误差。具体而言，这种划分方法可以在一定程度上避免系统输出耦合现象<sup>[4]</sup>。

### 1.3.1 INDI 姿态控制架构

对于大部分无人机姿态运动控制，常采用基于欧拉角参数化的局部线性化方法，滚转角、俯仰角和偏航角三个通道分别作为控制目标单独控制，使得无人机的旋转过程被线性化处理。但是不同于位置动态方程(??)，欧拉角动态方程(??)将三个姿态通道耦合在一起。实际上，无人机的姿态属于  $SO(3)$  上的流形结构，任何姿态变化都被视为流形上的测地线运动。而传统的局部线性化跟踪姿态的方法相当于在流形的切空间进行局部线性逼近，这种线性化处理方式导致了姿态轨迹生成过程中的不连续性，丢失了流形的曲率信息，所以在机动飞行时会导致姿态跟踪误差变大。因此本文参考文献[5] 在  $SO(3)$  上考虑姿态误差的表示。

由于旋转矩阵  $\mathbf{R} \in SO(3)$ ，姿态欧拉角  $\boldsymbol{\eta} \in \mathbb{R}^3$ ，因此定义符号  $[\cdot]_{\times}$  表示映射关系  $\mathbb{R}^3 \rightarrow SO(3)$ ， $(\cdot)^{\vee}$  表示对应的逆映射  $SO(3) \rightarrow \mathbb{R}^3$ 。对于任意姿态  $\boldsymbol{\eta} = [\varphi \ \theta \ \psi]^T$ ，对应的矩阵形式可以表示为：

$$\mathbf{R} = [\boldsymbol{\eta}]_{\times} = \begin{bmatrix} 0 & -\psi & \theta \\ \psi & 0 & -\varphi \\ -\theta & \varphi & 0 \end{bmatrix} \quad (1-22)$$

假设期望姿态为  $\boldsymbol{\eta}_d$ ，当前姿态为  $\boldsymbol{\eta}$ ，对应的矩阵形式分别为  $\mathbf{R}_d$  和  $\mathbf{R}$ 。则在  $SO(3)$  上的姿态误差可以表示为：

$$\mathbf{e}_R = \frac{1}{2}(\mathbf{R}_d^T \mathbf{R} - \mathbf{R}^T \mathbf{R}_d)^{\vee} \in \mathbb{R}^3 \quad (1-23)$$

因此期望的姿态欧拉角变化率可以表示为：

$$\dot{\eta}_1 = -\mathbf{K}_R \mathbf{e}_R \quad (1-24)$$

其中  $\mathbf{K}_R = \text{diag}(K_{R\varphi}, K_{R\theta}, K_{R\psi})$  为姿态误差的正定增益矩阵。式(1-24)所示的作差方式类似于反馈信号减去给定信号，所以需要加负号。

为提高系统的响应速度，确保快速跟踪期望姿态，引入前馈控制策略：

$$\dot{\eta}_2 = \frac{d\eta_d}{dt} \quad (1-25)$$

结合式(??)、式(1-24)和式(1-25)，期望的机体角速度  $\omega_d^b$  可以表示为：

$$\omega_d^b = \mathbf{Q}^{-1}(\dot{\eta}_1 + \dot{\eta}_2) \quad (1-26)$$

接下来考虑与动力学相关的角速度动态，包含了可控力矩与不可控力矩。如前文所述，DFUAV 姿态子系统的控制输入为四片相互独立的控制舵面的偏转角  $\delta_i$ ，由四个偏转角度的组合产生用于控制机体姿态的俯仰力矩、滚转力矩和偏航力矩，因此姿态子系统中存在执行机构的冗余，存在过驱动的问题。引入虚拟控制输入  $\boldsymbol{\nu} \in \mathbb{R}^3$  将姿态控制算法与控制分配算法分开：

$$\boldsymbol{\nu} = \begin{bmatrix} \nu_x \\ \nu_y \\ \nu_z \end{bmatrix} = \mathbf{B} \begin{bmatrix} \delta_1 \\ \delta_2 \\ \delta_3 \\ \delta_4 \end{bmatrix} \quad (1-27)$$

其中矩阵  $\mathbf{B}$  表示四片独立控制舵面偏转角到三维虚拟控制输入的映射矩阵，有  $\mathbf{B} \in \mathbb{R}^{3 \times 4}$ 。

结合式(??)、式(??)、式(1-14)和式(1-27)，可以将可控的控制舵面力矩重写为模块化形式：

$$(\mathbf{J}^b)^{-1} \mathbf{M}_{vane}^b = \mathbf{H}_{vane} \boldsymbol{\nu} \Omega^2 \quad (1-28)$$

其中

$$\mathbf{H}_{vane} \triangleq k_\delta k_f^2 \begin{bmatrix} 2J_x^{-1} l_1 & & \\ & 2J_y^{-1} l_1 & \\ & & 4J_z^{-1} l_2 \end{bmatrix} \in \mathbb{R}^{3 \times 3} \quad (1-29)$$

$$\mathbf{B} = \begin{bmatrix} -0.5 & 0 & 0.5 & 0 \\ 0 & -0.5 & 0 & 0.5 \\ 0.25 & 0.25 & 0.25 & 0.25 \end{bmatrix} \quad (1-30)$$

经过式(1-28)的模块化处理，由原本的从控制舵面角度到控制力矩的映射转变为了从控制舵面角度到虚拟控制输入的映射。在此过程中，映射矩阵剔除了所有与模型相关参数项。

对于可控的陀螺力矩  $\mathbf{M}_{gyro}^b$ ，根据式(??)和式??，可将其模块化表示为：

$$(\mathbf{J}^b)^{-1} \mathbf{M}_{gyro}^b = \mathbf{H}_{gyro}(\boldsymbol{\omega}^b) \boldsymbol{\Omega} \quad (1-31)$$

其中

$$\mathbf{H}_{gyro}(\boldsymbol{\omega}^b) \triangleq J_{fan} [-J_x^{-1} q \quad J_y^{-1} p \quad 0]^T \in \mathbb{R}^3 \quad (1-32)$$

对于不可控的气动力矩  $\mathbf{M}_a^b$ ，将其与式(??)中的非线性项  $(\mathbf{J}^b \boldsymbol{\omega}^b \times \boldsymbol{\omega}^b)$  一同处理，表示为与地面坐标系下的速度  $\mathbf{V}^e$ 、姿态  $\boldsymbol{\eta}$ 、环境风速  $\mathbf{W}^e$  和机体角速度  $\boldsymbol{\omega}^b$  有关的函数：

$$(\mathbf{J}^b)^{-1} (\mathbf{M}_a^b + \mathbf{J} \boldsymbol{\omega}^b \times \boldsymbol{\omega}^b) \triangleq \mathbf{L}(\mathbf{V}^e, \boldsymbol{\eta}, \mathbf{W}^e, \boldsymbol{\omega}^b) \in \mathbb{R}^3 \quad (1-33)$$

结合式(1-28)、式(1-31)和式(1-33)，角速度动态方程可以被模块化表示为：

$$\dot{\boldsymbol{\omega}}^b = \mathbf{H}_{vane} \boldsymbol{\nu} \Omega^2 + \mathbf{H}_{gyro}(\boldsymbol{\omega}^b) \boldsymbol{\Omega} + \mathbf{L}(\mathbf{V}^e, \boldsymbol{\eta}, \mathbf{W}^e, \boldsymbol{\omega}^b) \quad (1-34)$$

可以直接在该模型的基础上应用 INDI，而无需考虑控制分配问题。根据本章第一节介绍的 INDI 理论，为了得到  $\dot{\boldsymbol{\omega}}^b$  的增量表达式，对公式(1-34)在其工作点处 (使用下标  $(\cdot)_0$  表示) 作一阶泰勒展开：

$$\begin{aligned} \dot{\boldsymbol{\omega}}^b &= \mathbf{H}_{vane} \boldsymbol{\nu} \Omega^2 + \mathbf{H}_{gyro}(\boldsymbol{\omega}^b) \boldsymbol{\Omega} + \mathbf{L}(\mathbf{V}^e, \boldsymbol{\eta}, \mathbf{W}^e, \boldsymbol{\omega}^b) \\ &= \mathbf{H}_{vane} \boldsymbol{\nu}_0 \Omega_0^2 + \mathbf{H}_{gyro}(\boldsymbol{\omega}_0^b) \boldsymbol{\Omega}_0 + \mathbf{L}_0 \\ &\quad + \left. \frac{\partial(\mathbf{H}_{vane} \boldsymbol{\nu} \Omega^2)}{\partial \boldsymbol{\nu}} \right|_{\boldsymbol{\nu}=\boldsymbol{\nu}_0} (\boldsymbol{\nu} - \boldsymbol{\nu}_0) + \left[ \left. \frac{\partial(\mathbf{H}_{vane} \boldsymbol{\nu} \Omega^2)}{\partial \Omega} \right|_{\Omega=\Omega_0} + \left. \frac{\partial(\mathbf{H}_{gyro} \boldsymbol{\Omega})}{\partial \boldsymbol{\Omega}} \right|_{\boldsymbol{\Omega}=\boldsymbol{\Omega}_0} \right] (\boldsymbol{\Omega} - \boldsymbol{\Omega}_0) \\ &\quad + \left[ \left. \frac{\partial(\mathbf{H}_{gyro} \boldsymbol{\Omega})}{\partial \boldsymbol{\omega}^b} \right|_{\boldsymbol{\omega}^b=\boldsymbol{\omega}_0^b} + \left. \frac{\partial \mathbf{L}}{\partial \boldsymbol{\omega}^b} \right|_{\boldsymbol{\omega}^b=\boldsymbol{\omega}_0^b} \right] (\boldsymbol{\omega}^b - \boldsymbol{\omega}_0^b) \\ &\quad + \left. \frac{\partial \mathbf{L}}{\partial \mathbf{V}^e} \right|_{\mathbf{V}^e=\mathbf{V}_0^e} (\mathbf{V}^e - \mathbf{V}_0^e) + \left. \frac{\partial \mathbf{L}}{\partial \boldsymbol{\eta}} \right|_{\boldsymbol{\eta}=\boldsymbol{\eta}_0} (\boldsymbol{\eta} - \boldsymbol{\eta}_0) + \left. \frac{\partial \mathbf{L}}{\partial \mathbf{W}^e} \right|_{\mathbf{W}^e=\mathbf{W}_0^e} (\mathbf{W}^e - \mathbf{W}_0^e) \end{aligned} \quad (1-35)$$

展开式中的第一项  $\mathbf{H}_{vane}\boldsymbol{\nu}_0\Omega_0^2 + \mathbf{H}_{gyro}(\boldsymbol{\omega}_0^b)\Omega_0 + \mathbf{L}_0$  等价于工作点处的角加速度，可以由机载 IMU 测量的角速度经过差分滤波得到：

$$\dot{\boldsymbol{\omega}}_0^b = \mathbf{H}_{vane}\boldsymbol{\nu}_0\Omega_0^2 + \mathbf{H}_{gyro}(\boldsymbol{\omega}_0^b)\Omega_0 + \mathbf{L}_0 \quad (1-36)$$

展开式中的其他项是关于系统内部状态变量  $(\mathbf{V}^e, \boldsymbol{\eta}, \boldsymbol{\omega}^b)$ 、系统控制输入  $(\boldsymbol{\nu}, \Omega)$  和外部扰动  $\mathbf{W}^e$  的一阶偏导数项。根据本章第一小节介绍的时间尺度分离法则，认为在角加速度的输入信号的时间尺度内，系统内部状态变量包括速度、姿态和机体角速度的变化是缓慢的，因此可以将这些状态变量的一阶偏导数项视为 0：

$$\begin{cases} \mathbf{V}^e - \mathbf{V}_0^e \approx 0 \\ \boldsymbol{\eta} - \boldsymbol{\eta}_0 \approx 0 \\ \boldsymbol{\omega}^b - \boldsymbol{\omega}_0^b \approx 0 \end{cases} \quad (1-37)$$

在无人机控制中，执行机构(如电机、舵机等)可在数十毫秒的时间尺度内完成控制指令的执行，产生所需的力与力矩。而机体的速度、姿态等状态变量受空气动力阻尼、质量惯性等因素的制约，其动态过程一般落后于执行机构响应 1-2 个数量级，因此上述假设是合理的。此外，对于外部环境风速扰动  $\mathbf{W}^e$  的变化是未知并且难以预测的，因此假定  $(\mathbf{W}^e - \mathbf{W}_0^e \approx 0)$ 。但此假设并没有排除外部环境风速扰动对系统的影响，缓慢变化的自然风对姿态造成的影响已被考虑在  $\mathbf{L}_0$  中。而应对快速变化的风速扰动则取决于所设计的姿态控制器的鲁棒性。

其余关于系统控制输入的偏导数项如下：

$$\begin{aligned} & \left. \frac{\partial(\mathbf{H}_{vane}\boldsymbol{\nu}\Omega^2)}{\partial\boldsymbol{\nu}} \right|_{\boldsymbol{\nu}=\boldsymbol{\nu}_0} (\boldsymbol{\nu} - \boldsymbol{\nu}_0) = \mathbf{H}_{vane}\Omega_0^2(\boldsymbol{\nu} - \boldsymbol{\nu}_0) \\ & \left[ \frac{\partial(\mathbf{H}_{vane}\boldsymbol{\nu}\Omega^2)}{\partial\Omega} + \frac{\partial(\mathbf{H}_{gyro}\Omega)}{\partial\Omega} \right] \Big|_{\Omega=\Omega_0} (\Omega - \Omega_0) = [2\mathbf{H}_{vane}\boldsymbol{\nu}_0\Omega_0 + \mathbf{H}_{gyro}(\boldsymbol{\omega}_0^b)](\Omega - \Omega_0) \end{aligned} \quad (1-38)$$

因此，角速度动态的一阶泰勒展开式(1-35)近似可以表示为：

$$\dot{\boldsymbol{\omega}}^b = \dot{\boldsymbol{\omega}}_0^b + \mathbf{H}_{vane}\Omega_0^2(\boldsymbol{\nu} - \boldsymbol{\nu}_0) + [2\mathbf{H}_{vane}\boldsymbol{\nu}_0\Omega_0 + \mathbf{H}_{gyro}(\boldsymbol{\omega}_0^b)](\Omega - \Omega_0) \quad (1-39)$$

值得一提的是，风扇转速  $\Omega$  作为速度控制的输入信号并不直接参与姿态控制。简化后的角速度动态(即式(1-21))中与  $\Omega$  相关的项均与陀螺力矩  $\mathbf{M}_{gyro}^b$  有关，将由下文设计的陀螺力矩补偿算法补偿。因此  $\Omega$  不被视为姿态控制中输入信号的一部分，可以将  $\Omega$

作为在角速度动态中的已知参数，该参数可以从速度环设计中得到。为了简便表示，采用以下记号：

$$\mathbf{T}(\Omega) = [2\mathbf{H}_{vane}\boldsymbol{\nu}_0\Omega_0 + \mathbf{H}_{gyro}(\boldsymbol{\omega}_0^b)](\Omega - \Omega_0) \in \mathbb{R}^3 \quad (1-40)$$

将式(1-40)代入式(1-39)，可以得到角速度  $\boldsymbol{\omega}^b$  动态的增量表达式：

$$\begin{aligned} \dot{\boldsymbol{\omega}}^b &= \dot{\boldsymbol{\omega}}_0^b + \mathbf{H}_{vane}\Omega_0^2(\boldsymbol{\nu} - \boldsymbol{\nu}_0) + \mathbf{T}(\Omega) \\ &= \begin{bmatrix} \dot{p}_0 \\ \dot{q}_0 \\ \dot{r}_0 \end{bmatrix} + k_\delta k_f^2 \Omega_0^2 \begin{bmatrix} 2J_x^{-1}l_1 & & \\ & 2J_y^{-1}l_1 & \\ & & 4J_z^{-1}l_2 \end{bmatrix} \left( \begin{bmatrix} \nu_x \\ \nu_y \\ \nu_z \end{bmatrix} - \begin{bmatrix} \nu_{x0} \\ \nu_{y0} \\ \nu_{z0} \end{bmatrix} \right) + \begin{bmatrix} T_x(\Omega) \\ T_y(\Omega) \\ T_z(\Omega) \end{bmatrix} \end{aligned} \quad (1-41)$$

INDI 通过对式(1-41)进行求逆，来得到期望的虚拟控制输入的增量表达式：

$$\begin{aligned} \boldsymbol{\nu}_d &= (\mathbf{H}_{vane}\Omega_0^2)^{-1} \left\{ \boldsymbol{\mu} - [\dot{\boldsymbol{\omega}}_0^b + \mathbf{T}(\Omega_d)] \right\} + \boldsymbol{\nu}_0 \\ &= \frac{1}{k_\delta k_f^2 \Omega_0^2} \begin{bmatrix} 2J_x^{-1}l_1 & & \\ & 2J_y^{-1}l_1 & \\ & & 4J_z^{-1}l_2 \end{bmatrix}^{-1} \left\{ \begin{bmatrix} \mu_x \\ \mu_y \\ \mu_z \end{bmatrix} - \left( \begin{bmatrix} \dot{p}_0 \\ \dot{q}_0 \\ \dot{r}_0 \end{bmatrix} + \begin{bmatrix} T_x(\Omega_d) \\ T_y(\Omega_d) \\ T_z(\Omega_d) \end{bmatrix} \right) \right\} + \begin{bmatrix} \nu_{x0} \\ \nu_{y0} \\ \nu_{z0} \end{bmatrix} \end{aligned} \quad (1-42)$$

其中，虚拟输入项  $\boldsymbol{\nu}$  被  $\boldsymbol{\nu}_d$  替换，表明是期望的输入。类似地， $\Omega$  被替换为  $\Omega_d$ 。

将式(1-42)代入式(1-41)，得到线性化的角速度动态方程：

$$\dot{\boldsymbol{\omega}}^b = \boldsymbol{\mu} \quad (1-43)$$

最终，通过以下比例反馈律，可以得到稳定的角速度子系统：

$$\boldsymbol{\mu} = \mathbf{K}_\omega(\boldsymbol{\omega}_d^b - \boldsymbol{\omega}^b) \quad (1-44)$$

将式(1-44)代入式(1-43)中可以得到：

$$\dot{\boldsymbol{\omega}}^b = \mathbf{K}_\omega(\boldsymbol{\omega}_d^b - \boldsymbol{\omega}^b) \quad (1-45)$$

其中  $\mathbf{K}_\omega = \text{diag}(K_{\omega p}, K_{\omega q}, K_{\omega r})$  为正反馈增益矩阵。

基于 INDI 的控制思想已经把非线性角速度动力学方程(1-34)简化为了一个一阶积分系统(1-43)，因此采用单一的比例控制器就可以确保闭环系统的稳定性，只需要保证反馈增益为正，而不管其具体取值。这进一步提供了一个指导，可以分别确定闭环系统

的静态平衡点和动态性能。因此结合式(1-44)，可以将式(1-42)可以分为两部分：

$$\boldsymbol{\nu}_d = \boldsymbol{\nu}_i + \boldsymbol{\nu}_f \quad (1-46)$$

其中  $\boldsymbol{\nu}_i$  定义为 INDI 输入， $\boldsymbol{\nu}_f$  定义为反馈输入，具体形式为：

$$\begin{aligned} \boldsymbol{\nu}_i &= -(\mathbf{H}_{vane}\Omega_0^2)^{-1}[\dot{\boldsymbol{\omega}}_0^b + \mathbf{T}(\Omega_d)] + \boldsymbol{\nu}_0 \\ &= -\frac{1}{k_\delta k_f^2 \Omega_0^2} \begin{bmatrix} 2J_x^{-1}l_1 & & \\ & 2J_y^{-1}l_1 & \\ & & 4J_z^{-1}l_2 \end{bmatrix}^{-1} \left( \begin{bmatrix} \dot{p}_0 \\ \dot{q}_0 \\ \dot{r}_0 \end{bmatrix} + \begin{bmatrix} T_x(\Omega_d) \\ T_y(\Omega_d) \\ T_z(\Omega_d) \end{bmatrix} \right) + \begin{bmatrix} \nu_{x0} \\ \nu_{y0} \\ \nu_{z0} \end{bmatrix} \end{aligned} \quad (1-47)$$

$$\begin{aligned} \boldsymbol{\nu}_f &= (\mathbf{H}_{vane}\Omega_0^2)^{-1} \mathbf{K}_\omega (\boldsymbol{\omega}_d^b - \boldsymbol{\omega}^b) \\ &= \frac{1}{k_\delta k_f^2 \Omega_0^2} \begin{bmatrix} 2J_x^{-1}l_1 & & \\ & 2J_y^{-1}l_1 & \\ & & 4J_z^{-1}l_2 \end{bmatrix}^{-1} \begin{bmatrix} K_{\omega p} & & \\ & K_{\omega q} & \\ & & K_{\omega r} \end{bmatrix} \left( \begin{bmatrix} \dot{p}_d \\ \dot{q}_d \\ \dot{r}_d \end{bmatrix} - \begin{bmatrix} \dot{p} \\ \dot{q} \\ \dot{r} \end{bmatrix} \right) \end{aligned} \quad (1-48)$$

INDI 采用增量控制策略，将未建模动态力矩项  $\mathbf{M}_a^b$  和角速度动力学耦合项  $\mathbf{J}\boldsymbol{\omega}^b \times \boldsymbol{\omega}^b$  造成的影响放在了  $\dot{\boldsymbol{\omega}}_0^b$  中处理。在每一个工作点计算的 INDI 输入都是基于上一个工作点的 INDI 输入加上模型误差、阵风扰动以及风扇转速输入对相邻两个工作点之间的角加速度影响。因此，模型误差不会随着时间进行累计，而是在每一个工作点实时补偿。而系统的稳定性由基于误差的角速度反馈来保证。

根据上述分析，INDI 输入的计算依赖于机体的角加速度值  $\dot{\boldsymbol{\omega}}_0^b$ ，在现有条件下，该值可以通过对机体陀螺仪测量的相邻两个采样点的角速度进行一阶差分得到：

$$\dot{\boldsymbol{\omega}}^b(k) = \frac{[\boldsymbol{\omega}^b(k) - \boldsymbol{\omega}^b(k-1)]}{\Delta t} \quad (1-49)$$

其中  $\Delta t$  是陀螺仪传感器的采样时间， $k$  为传感器的采样点，泰勒展开式中的工作点一般选取为当前的采样时刻。因此 INDI 也被称为基于传感器的方法。

在飞行控制系统的工程实现中，为了提高姿态控制器的响应速度和控制精度， $\Delta t$  应该在条件允许的情况下尽可能小。但是传感器测量得到的角速度原始值会受到机体震动和外部扰动等因素的影响，难免会有噪声，对原始值进行差分运算后会放大这种噪声的影响，采样时间越小，噪声影响越大。因此在实际应用中，需要对角速度进行滤波处

理。但是滤波处理就会在角加速度的计算中引入延迟，由于 INDI 是将角速度动力学方程在工作点处进行一阶泰勒展开，所以需要保证展开式(1-41)中所有下标为 0 的项都位于同一工作点。一种方法是对角加速度进行预测，如采用卡尔曼滤波器对其进行预测来补偿相位滞后，但是这种方法需要对角加速度进行额外的建模，并且角速度收到的的干扰也无法预测。或者采用多个加速度计数据融合来直接测量角加速度，但是这种方式会导致机载航电系统设计过于复杂，并且加速度计受机体震动影响也较大，多传感器数据融合的精度难以有效保证。

因此，本文针对所有下标为 0 的项进行同步滤波处理，除了角加速度项外，还包括虚拟控制输入项和风扇转速输入项。文献[6-7] 通过四旋翼飞行实验表明这种处理方式的可行性。考虑到陀螺仪传感器的高频噪声影响，本文实验使用的滤波器是二阶巴特沃斯 (Butterworth) 低通滤波器，滤波器截止频率的具体数值应该根据飞行过程中信号的频率成分、带宽和噪声水平来综合确定。对于本文使用的 DFUAV，滤波器截止频率为 30Hz。为加以区分，所有下标为 0 的项经过滤波器滤波后，使用下标  $(.)_s$  表示。

### 1.3.2 陀螺力矩补偿

在基于 INDI 的控制架构中，陀螺力矩  $M_{gyro}^b$  分量蕴含在了  $T(\Omega)$  中 (见式(1-40))，该力矩的存在会在姿态发生改变时影响姿态跟踪精度。因此，为了消除陀螺力矩对姿态控制的影响，需要在 INDI 输入  $\nu_i$  中对其进行补偿。由于姿态子系统的控制输入为控制舵面的偏转角度，并且根据式(??)可知陀螺力矩的机理已十分明确。所以可以将陀螺力矩对应的值等价转化为虚拟控制输入量，然后在  $\nu_i$  中减去该分量，即可对陀螺力矩造成的负面影响进行补偿。

根据式(??)、(??)和式(1-14)，在现有条件下可用于计算的控制舵面力矩可表示为：

$$M_{vane}^b = k_\delta k_f^2 \Omega^2 \begin{bmatrix} -l_1 & 0 & l_1 & 0 \\ 0 & -l_1 & 0 & l_1 \\ l_2 & l_2 & l_2 & l_2 \end{bmatrix} \delta \quad (1-50)$$

将式(1-27)代入式(1-50)可得：

$$M_{vane}^b = k_\delta k_f^2 \Omega^2 \begin{bmatrix} -l_1 & 0 & l_1 & 0 \\ 0 & -l_1 & 0 & l_1 \\ l_2 & l_2 & l_2 & l_2 \end{bmatrix} B^\dagger \nu \quad (1-51)$$

根据式(1-12)以及式(1-30)中矩阵  $\mathbf{B}$  的值，可以计算得到  $\mathbf{B}^\dagger$  为：

$$\mathbf{B}^\dagger = \begin{bmatrix} -1 & 0 & 1 \\ 0 & -1 & 1 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix} \quad (1-52)$$

如果直接计算与陀螺力矩等效的控制舵面的偏转角度，那么就有可能在实际控制中出现控制舵面的偏转角度超过其可行范围的情况。这是因为与陀螺力矩等效的控制舵面的偏转角度未经过控制分配算法的优化分配，导致期望的控制效果与实际不符。因此，此处计算与陀螺力矩等效的虚拟控制输入以便于 INDI 输入  $\nu_i$  一起参与优先级控制分配。

令式(??)所描述的陀螺力矩和式(1-51)描述的控制舵面力矩相等，可以得到与陀螺力矩等效的虚拟控制输入  $\nu_{gyro}$ ：

$$\begin{aligned} \mathbf{M}_{gyro}^b &= \mathbf{M}_{vane}^b \\ \Rightarrow J_{fan}\Omega \begin{bmatrix} -q \\ p \\ 0 \end{bmatrix} &= k_\delta k_f^2 \Omega^2 \begin{bmatrix} -l_1 & 0 & l_1 & 0 \\ 0 & -l_1 & 0 & l_1 \\ l_2 & l_2 & l_2 & l_2 \end{bmatrix} \mathbf{B}^\dagger \nu_{gyro} \\ \Rightarrow \nu_{gyro} &= \frac{J_{fan}}{k_\delta k_f^2 \Omega} \mathbf{B} \begin{bmatrix} -l_1 & 0 & l_1 & 0 \\ 0 & -l_1 & 0 & l_1 \\ l_2 & l_2 & l_2 & l_2 \end{bmatrix}^\dagger \begin{bmatrix} -q \\ p \\ 0 \end{bmatrix} \end{aligned} \quad (1-53)$$

因此，加入陀螺力矩补偿后的 INDI 输入  $\nu_i$  为：

$$\nu_i = -(\mathbf{H}_{vane}\Omega_0^2)^{-1}[\dot{\omega}_0^b + \mathbf{T}(\Omega_d)] + \nu_0 - \nu_{gyro} \quad (1-54)$$

其中  $\nu_{gyro}$  表达式中的风扇转速项  $\Omega$  和角速度项  $\omega^b$  都需要同步滤波。

### 1.3.3 优先级控制分配

由于 DFUAV 的特殊构型，飞行过程中由期望控制输入计算出的控制舵面偏转角很可能超出其物理约束，所以底层设计用于解决控制分配问题。该问题可以描述为对于给定的三维期望虚拟控制输入  $\nu_d$ ，找到一组四片控制舵面的偏转角度在容许控制集合的范围内  $\delta_d \in \mathbf{U}$  使得  $\nu_d = \mathbf{B}\delta_d$ ，或者在无法找到这样一组解的情况下，需要找到一组最

优解使得分配误差  $\boldsymbol{\nu}_d - \mathbf{B}\boldsymbol{\delta}_d$  最小化。容许控制集合  $\mathbf{U}$  由下式定义：

$$\mathbf{U} = \{\boldsymbol{\delta} \in \mathbb{R}^4 | -\delta_m \leq \delta_i \leq \delta_m, i = 1, 2, 3, 4\} \quad (1-55)$$

对应的，虚拟控制输入  $\boldsymbol{\nu}$  的可达集合  $\Phi$  表示为：

$$\Phi = \{\boldsymbol{\nu} \in \mathbb{R}^3 | \boldsymbol{\nu} = \mathbf{B}\boldsymbol{\delta}, \boldsymbol{\delta} \in \mathbf{U}\} \quad (1-56)$$

其中  $\mathbf{B}$  由式(1-30)给出。

为了实现控制分配的目标，广泛采用的方法是直接计算矩阵  $\mathbf{B}$  的伪逆矩阵  $\mathbf{B}^\dagger$ ，然后计算  $\boldsymbol{\delta}_d$ ：

$$\boldsymbol{\delta}_d = \mathbf{B}^\dagger \boldsymbol{\nu}_d \quad (1-57)$$

使用伪逆方法的优点是计算简单，易于理解。但是在工作点的约束边界区，即使  $\boldsymbol{\nu}_d$  保持在可达集合内，伪逆法计算出的  $\boldsymbol{\delta}_d$  中的部分分量也可能会超出约束边界，即  $\boldsymbol{\delta} \notin \mathbf{U}$ 。在这种情况下，伪逆法会计算出一个基于某种性能指标的次最优解，次最优解可能有多种选择，所以次最优解之间亦有优劣之分。伪逆法没有对这些解进行优先级排序，因此无法保证次最优解的唯一性。

为了弥补伪逆方法的不足，采用优先级控制分配方法，将期望的虚拟控制输入进行矢量分解划分各自的优先级，优先满足高优先级的控制输入分量。式(1-46)已将期望的虚拟控制输入  $\boldsymbol{\nu}_d$  分为两部分，即  $\boldsymbol{\nu}_i$  和  $\boldsymbol{\nu}_f$ ，其中  $\boldsymbol{\nu}_i$  是 INDI 输入， $\boldsymbol{\nu}_f$  是反馈输入。根据本节开篇时的分析，为优先补偿系统的非线性未建模动态力矩，视  $\boldsymbol{\nu}_i$  为高优先级控制输入分量，而  $\boldsymbol{\nu}_f$  为低优先级控制输入分量。考虑到控制舵面的物理约束，期望的虚拟控制输入应该被限制在其可达集合内。在本文中，假设在适当的外部扰动下，无人机运动过程中总能满足高优先级分量的可达。即：

$$\boldsymbol{\nu}_i \in \Phi \quad (1-58)$$

如果由于控制舵面的饱和而使得  $\boldsymbol{\nu}_i$  超出可达集合，那么姿态将不受控制，闭环系统失去可控性。但是这种情况不在本文的讨论范围内。

由于总的虚拟控制输入  $\boldsymbol{\nu}_d$  是受到可达集合限制的，并且 INDI 输入分量  $\boldsymbol{\nu}_i$  假定总是可达的。这表明了总的虚拟控制输入中的另一部分分量，即反馈输入分量  $\boldsymbol{\nu}_f$ ，可能会超出可达集合的范围，因此需要对  $\boldsymbol{\nu}_f$  进行约束。使用比例因子  $\alpha$  将  $\boldsymbol{\nu}_f$  限制在可达

集合内，即：

$$\boldsymbol{\nu}_d = \boldsymbol{\nu}_i + \alpha \boldsymbol{\nu}_f, \quad \text{其中 } 0 \leq \alpha \leq 1 \quad (1-59)$$

因为  $\dot{\omega}^b$  仍由式(1-45)控制，所以保持  $\boldsymbol{\nu}_i$  不变并对  $\boldsymbol{\nu}_f$  进行缩放只会影响闭环系统的动态性能，而不会影响闭环系统的稳定性。

当虚拟控制输入可达时，比例因子  $\alpha$  等于 1，此时的优先级控制分配方法等价于伪逆法。当虚拟控制输入不可达时，比例因子  $\alpha$  小于 1，表明对低优先级分量  $\boldsymbol{\nu}_f$  进行截断，保证了高优先级分量  $\boldsymbol{\nu}_i$  的可达性。

综上所述，优先级控制分配算法可以通过解决以下线性优化问题来实现对高优先级分量的保持和低优先级分量的截断：

$$\begin{aligned} & \max_{\alpha, \boldsymbol{\delta}} \alpha \\ & \text{subject to } \begin{cases} \boldsymbol{\delta} \in \boldsymbol{U} \\ \boldsymbol{B}\boldsymbol{\delta} = \boldsymbol{\nu}_i + \alpha \boldsymbol{\nu}_f \\ 0 \leq \alpha \leq 1 \end{cases} \end{aligned} \quad (1-60)$$

在具体实践过程中，上述线性优化问题通过以下步骤进行求解：首先判断给定的  $\boldsymbol{\nu}_d = \boldsymbol{\nu}_i + \boldsymbol{\nu}_f$  是否可达，即  $\alpha$  等于 1 的情况。如果可达，直接使用伪逆法计算  $\boldsymbol{\delta}_d$  并返回最优解；如果不可达，即  $0 \leq \alpha < 1$ ，则可以使用单纯形法通过基变量变换逐步迭代逼近最优解。首先通过以下公式满足高优先级分量  $\boldsymbol{\nu}_i$ ：

$$\boldsymbol{\nu}_i = \boldsymbol{B}\boldsymbol{\delta}', \quad \boldsymbol{\delta}' = [\delta'_1 \quad \delta'_2 \quad \delta'_3 \quad \delta'_4]^T \in \boldsymbol{U} \quad (1-61)$$

如果  $\boldsymbol{\nu}_i$  不可达，则直接返回  $\boldsymbol{\delta}_d = \boldsymbol{\delta}'$ 。否则  $\boldsymbol{\nu}_i$  可达，接下来需要满足低优先级分量  $\boldsymbol{\nu}_f$ ，此时容许控制集合变为：

$$\boldsymbol{U}' = \{\boldsymbol{\delta} \in \mathbb{R}^4 \mid -\delta_m - \delta'_i \leq \delta_i \leq \delta_m - \delta'_i, i = 1, 2, 3, 4\} \quad (1-62)$$

可以通过下式计算低优先级分量对应的舵面偏转：

$$\boldsymbol{\nu}_f = \boldsymbol{B}\boldsymbol{\delta}'', \quad \boldsymbol{\delta}'' = [\delta''_1 \quad \delta''_2 \quad \delta''_3 \quad \delta''_4]^T \in \boldsymbol{U}' \quad (1-63)$$

最终得到期望舵面偏转角为：

$$\boldsymbol{\delta}_d = \boldsymbol{\delta}' + \boldsymbol{\delta}'' \quad (1-64)$$

式(1-60)的优化结果能够保证对于每一个由上层姿态控制算法计算出的期望虚拟控制输入，都有：

$$\boldsymbol{\nu}_d = \boldsymbol{\nu}_i + \alpha \boldsymbol{\nu}_f \in \Phi \quad (1-65)$$

这保证了  $\boldsymbol{\nu}_d$  的可达性以及次最优解的唯一性，区别仅是  $\alpha$  的取值不同。

值得注意的是，上述分解方式是人为规定的，在具体实践过程中还可以根据场景需求作其他种类的分解，比如定义分量的优先级和定义分解的维度等。除了 INDI 方法外，其他的控制方法包括串级 PID 控制律、ADRC 控制律<sup>[4]</sup>等都可以针对虚拟控制输入进行矢量分解。

图1-2所示的控制系统框图展示了上层设计采用 INDI 姿态控制方法，底层设计采用优先级控制分配方法的分层控制架构。

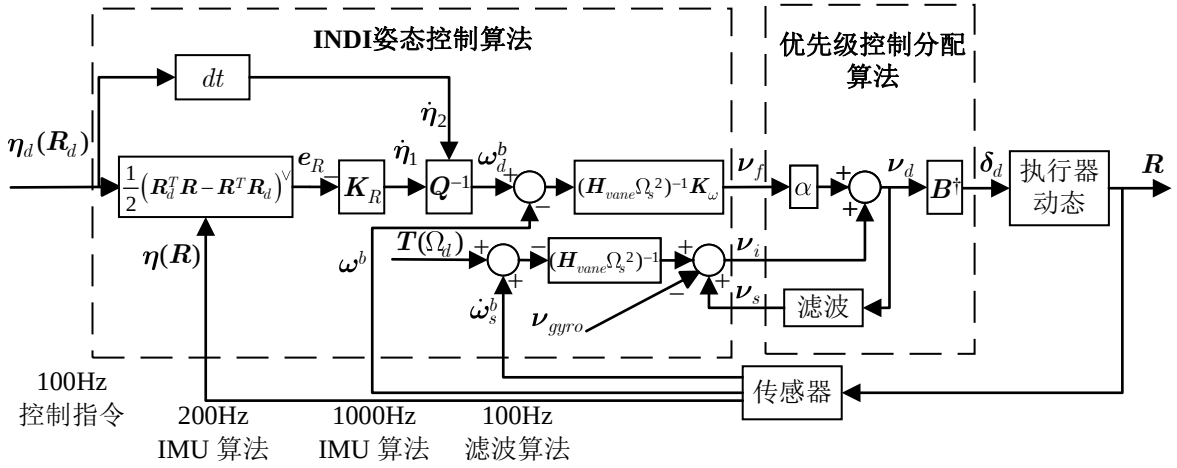


图 1-2 基于 INDI 和优先级控制分配法的姿态控制系统框图

## 1.4 实验和结果分析

为了验证提出的基于上层 INDI+ 底层优先级控制分配的姿态控制方法的性能，本小节进行了一系列仿真实验与实际飞行实验，包括 INDI 姿态控制方法对比串级 PID 姿态控制方法和优先级控制分配方法对比伪逆方法。作为对照实验的串级 PID 控制方法中，角度环采用单一的 P 控制器，角速度环采用 PI 控制器。实验过程中使用的 INDI 姿态控制器和串级 PID 姿态控制器的参数如下表1-1所示， $K_R$  表示角度环比例增益， $K_\omega$  表示角速度环比例增益， $I_\omega$  表示角速度环积分增益。为了保证实验过程中其他条件的一致性，在 INDI 控制对比 PID 控制的实验过程中，底层控制分配方法都采用伪逆法。而在优先级控制分配方法对比伪逆方法的实验过程中，上层姿态控制均采用 INDI 方法。

表 1-1 INDI 姿态控制器和 PID 姿态控制器的控制参数

| INDI 控制器参数符号   | 数值   | PID 控制器参数符号    | 数值   | PID 控制器参数符号    | 数值  |
|----------------|------|----------------|------|----------------|-----|
| $K_{R\varphi}$ | 5.5  | $K_{R\varphi}$ | 5.5  | $I_{\omega p}$ | 0.1 |
| $K_{R\theta}$  | 5.5  | $K_{R\theta}$  | 5.5  | $I_{\omega q}$ | 0.1 |
| $K_{R\psi}$    | 5.0  | $K_{R\psi}$    | 5.0  | $I_{\omega r}$ | 0.1 |
| $K_{\omega p}$ | 0.3  | $K_{\omega p}$ | 0.3  |                |     |
| $K_{\omega q}$ | 0.3  | $K_{\omega q}$ | 0.3  |                |     |
| $K_{\omega r}$ | 0.18 | $K_{\omega r}$ | 0.18 |                |     |

仿真实验平台为 MATLAB，DFUAV 的仿真模型相关参数参考表??，仿真模型中采用基于龙格-库塔 (Runge-Kutta) 法的 ode45 数值求解器求解 DFUAV 的非线性动态方程 (式(?))。实际飞行实验平台使用 2.1 节介绍的试验样机。为了提高数据的一致性并且减少风扇转速  $\Omega$  变化带来的耦合效应，需要保证姿态控制过程中的高度稳定，因此实验中采用了位置控制器，该控制器的设计方法将于第四章具体介绍。应当注意的是，位置控制器的加入并不影响姿态控制器的性能验证。悬停时风扇的旋转速度约为 7639 转/分钟。

为了体现所提出的姿态控制器的抗干扰性能，本文设计了一种主动产生扰动的方案。由于姿态动力学系统的输入信号为控制舵面的偏转角度，所以实验中通过在一些控制舵面上添加固定的偏置来引入干扰，如图1-3所示。此外，为了防止因加入固定偏置而导致控制舵面饱和的问题，当添加固定偏置时，对应控制舵面的最大偏转角度也应该同步调整。调整时也需要确保控制算法在正负方向上响应的对称性。例如在 1 号控制舵面上添加了  $5^\circ$  的固定偏置，那么 1 号控制舵面的最大偏转角度应该限制为  $\pm(\delta_m - 5^\circ)$ ，即  $\pm 35^\circ$ ，其他舵面的最大偏转角度保持  $\pm 40^\circ$  不变。

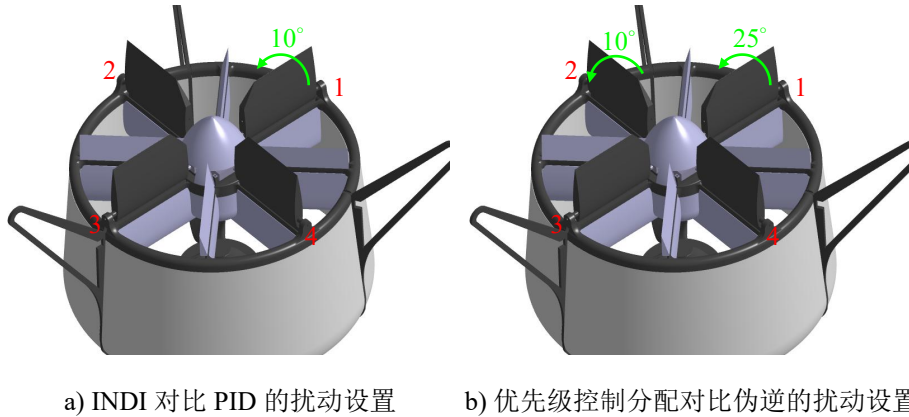


图 1-3 主动扰动实验设置

### 1.4.1 仿真实验

#### 1.4.1.1 INDI 对比 PID

本实验的目的是验证在 MATLAB 仿真环境中搭建的 INDI 控制器在有无干扰情况下的性能，采用 PID 姿态控制器作为实验对比。扰动的添加方式如图1-3a所示，在 1 号控制舵面上添加  $10^\circ$  的固定偏置，这会在机体上产生额外的负滚转力矩和正偏航力矩，1 号舵的偏转角度限幅被同时调整为  $\pm 30^\circ$ 。实验总时长为 6s，仿真步长为 0.01s。仅在滚转通道设置方波指令信号，俯仰和偏航通道的指令均设置为 0。

没有添加扰动时的实验对比结果如图1-4所示。根据实验结果可知，在滚转通道上，INDI 控制和 PID 控制都可以及时地跟踪期望姿态指令，但是相比于 PID 控制，INDI 控制器的超调量相对较小，并且收敛速度更快。对于俯仰和偏航通道，添加了陀螺力矩补偿的 INDI 控制器在滚转姿态变化时能够更好地抑制陀螺力矩的影响，从而提高了姿态控制的精度。

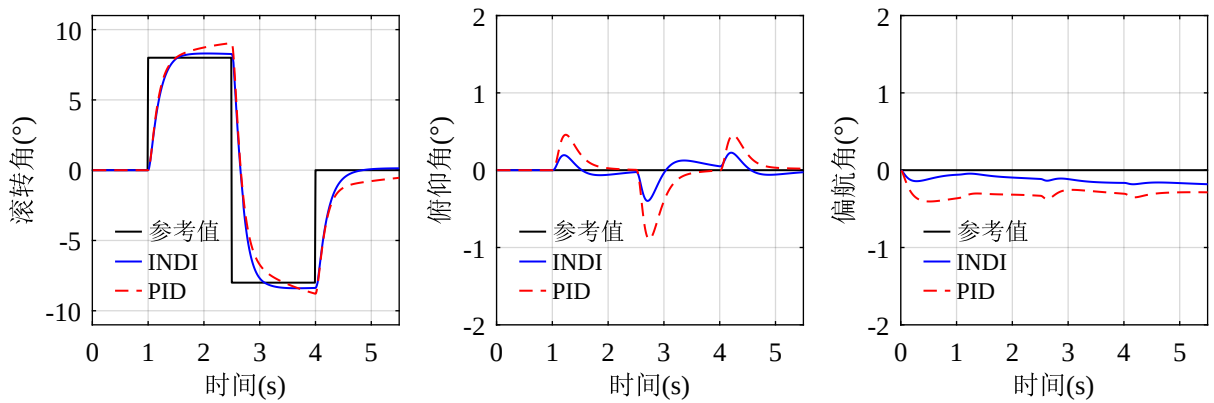


图 1-4 INDI 对比 PID 在无扰动情况下的仿真实验结果

添加干扰情况下的实验对比结果如图1-5所示，扰动在方波输入时同步添加。在滚转通道上，INDI 控制器在添加扰动后仍然能够保持较好的跟踪性能，似乎与未添加扰动情况下的姿态响应曲线没有区别，这是由于 INDI 控制器能够补偿未建模动态力矩  $M_a^b$ ，并且在每一个控制周期内都会实时补偿而不会积累误差。但是 PID 控制器依赖误差才能修正，存在未知扰动的情况下由于无法及时预测和补偿误差，所以表现出了明显的滞后与超调量，并且在 INDI 控制下的姿态达到稳态时，PID 控制下的姿态仍存在明显的偏差。在偏航通道上，INDI 对于扰动引起的偏航力矩的抑制能力同样明显优于 PID。

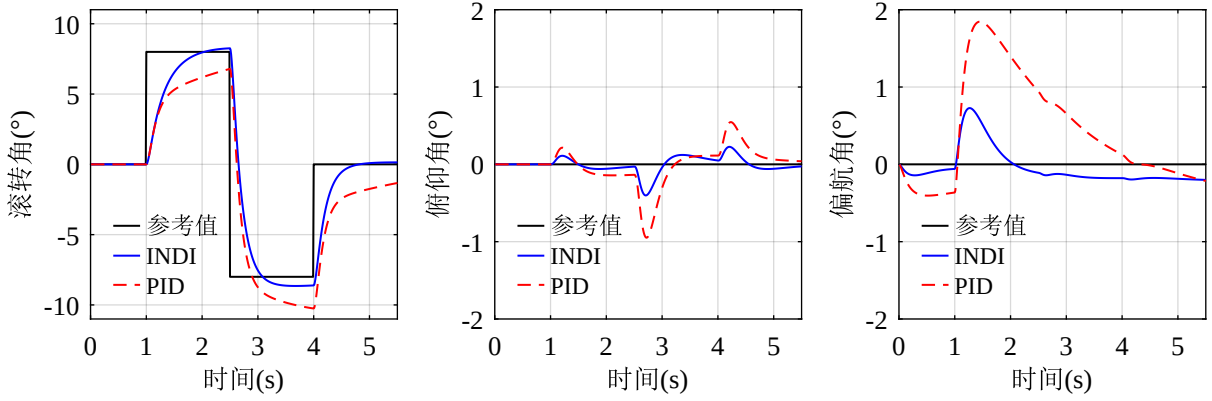


图 1-5 INDI 对比 PID 在有扰动情况下的仿真实验结果

#### 1.4.1.2 优先级控制分配对比伪逆

本实验用于验证设计的优先级控制分配算法在控制舵面饱和情况下的有效性，对比实验为前文提到的伪逆方法。扰动的添加方式如图1-3b所示，在1号控制舵面上添加 $25^\circ$ 的固定偏置，并且在2号控制舵面上添加 $10^\circ$ 的固定偏置，这会在机体上产生额外的负滚转力矩、负俯仰力矩和正偏航力矩。1号舵的偏转角度限幅被同时调整为 $\pm 15^\circ$ ，2号舵的偏转角度限幅被同时调整为 $\pm 30^\circ$ 。实验总时长为2s，仿真步长为0.01s。为了便于观察三个通道受扰动影响的情况，三个姿态通道的参考指令均设置为0，扰动在第1s时刻添加。

姿态通道的实验结果如图1-6所示。可以看出，三个姿态通道的误差变化情况都与扰动产生的力矩的变化情况相一致。但是以优先级控制分配算法作为底层分配算法时的姿态通道误差幅值明显比伪逆法更小，这是由于优先级控制分配算法优先满足了高优先级分量 $\nu_i$ ，使执行机构的变化能够及时地补偿未建模动态力矩的影响。这也说明了本文的划分优先级分量思路的正确性。

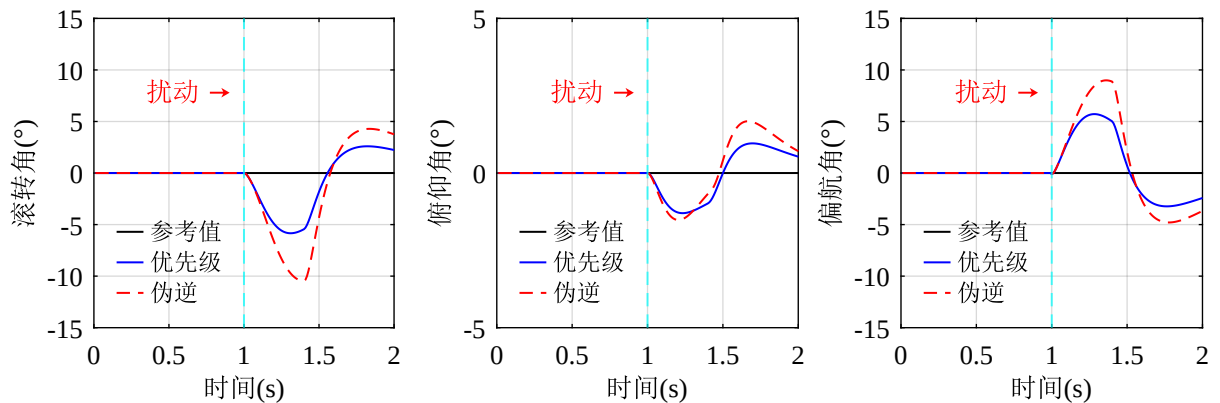


图 1-6 优先级控制分配对比伪逆法的姿态保持仿真实验结果

另一种可以直观感受控制分配方法优劣的方式是比较分配误差。伪逆方法的分配误差定义为：

$$e_{pinv} = \nu_d - B\delta_d \quad (1-66)$$

其中  $\delta_d$  是通过式(1-57)计算得到的，该值需要被限制在容许控制集合  $U$  内。

优先级控制分配算法的分配误差定义为  $e_i + e_f$ ，两个分量分别表示 INDI 分配误差和反馈分配误差。可以通过下式计算：

$$\begin{cases} e_i = \nu_i - B\delta' \\ e_f = \nu_f - B\delta'' \end{cases} \quad (1-67)$$

其中  $\delta'$  可以由式(1-61)计算得到， $\delta''$  可以由式(1-63)计算得到。

两种分配方法对应的分配误差如图1-7所示。可以看出，扰动添加的瞬间立刻在三个通道上产生分配误差。但是优先级控制分配算法的分配误差在扰动添加之后的短暂时间内迅速减小，而伪逆方法的分配误差则是不断增大一段时间才减小，且分配误差的幅值也明显大于优先级控制分配算法。说明本文使用的优先级控制分配算法提高了姿态控制的精度。

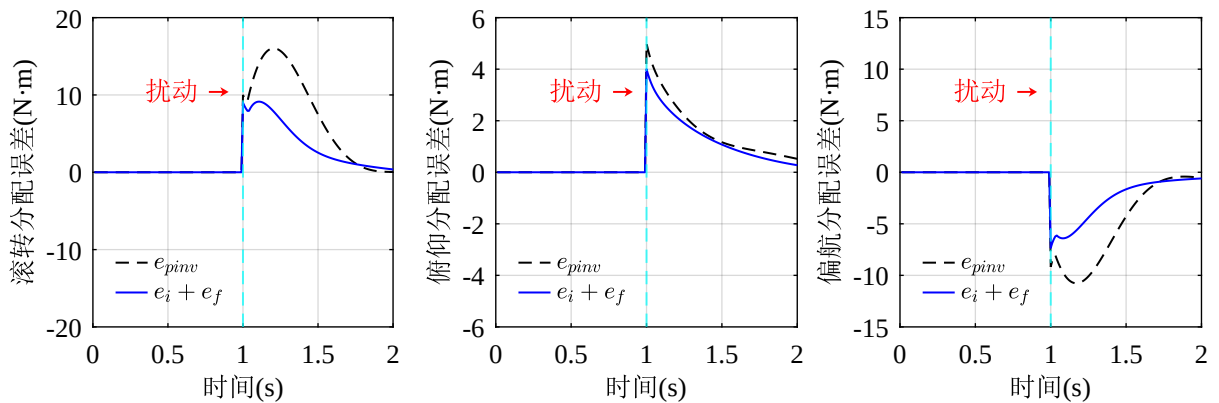


图 1-7 优先级控制分配对比伪逆法的分配误差仿真实验结果

图1-8展示了仿真实验中优先级分量各自对应的分配误差  $e_i$  和  $e_f$  的比较。从图中可以明显的看出 INDI 分配误差  $e_i$  始终保持为零，这表明分量  $\nu_i$  在实验过程中始终满足可达性约束，这是因为 INDI 输入分量位于高优先级，分配算法将优先满足该分量。而优先级控制分配的分配误差全由反馈分配误差  $e_f$  贡献，这表明了优先级控制分配算法由于控制舵面的饱和对反馈输入进行了截断。通过仿真实验验证了优先级控制分配算法的可行性和有效性。

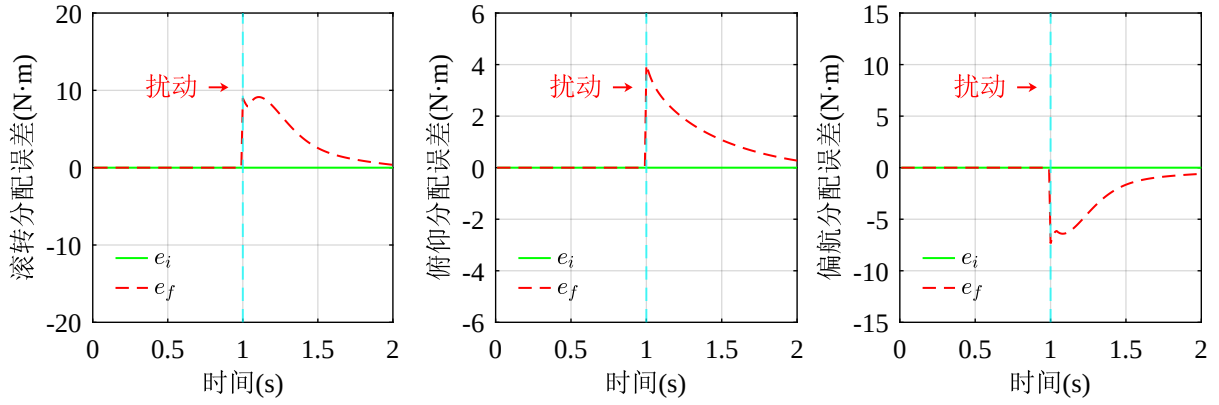


图 1-8 优先级分量各自的分配误差仿真实验结果

## 1.4.2 实际飞行实验

### 1.4.2.1 INDI 对比 PID

本实验的目的是使用第二章介绍的试验样机验证提出的基于 INDI 的姿态控制器在实际飞行中是否能够稳定的跟踪期望姿态以及抵抗外部干扰的能力。实验过程同样分为无扰动和有扰动两种情况。实验条件的设计与仿真实验中保持一致，包括扰动的添加方式(见图1-3a)和方波指令的输入等。实际控制器的步长为 0.01s。

无扰动和有扰动情况下的姿态控制实际飞行实验结果分别如图1-9和图1-10所示。从实验结果可以看出，在实际飞行情况下，INDI 姿态控制器跟踪期望姿态的能力同样优于 PID 姿态控制器。相比 PID，INDI 在滚转通道的超调量更小，上升时间更短，在俯仰和偏航通道上的姿态误差也更小。结果与仿真实验中相符。

在添加干扰的情况下，INDI 姿态控制器同样也表现出了更好的抗干扰能力，跟踪误差较小。而 PID 控制器则难以有效应对外部干扰，表现出了较大的跟踪误差。这进一步验证了 INDI 控制器在实际飞行中的有效性。

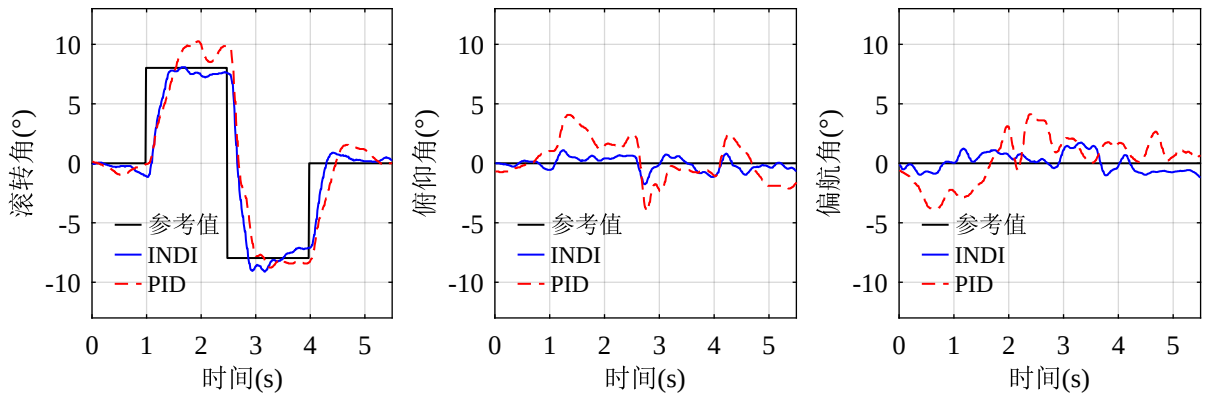


图 1-9 INDI 对比 PID 无扰动时的飞行实验结果

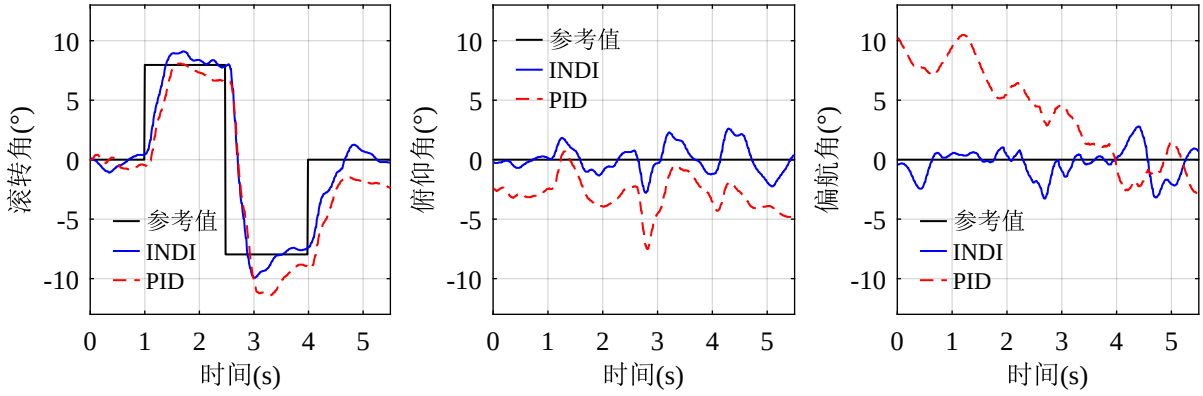


图 1-10 INDI 对比 PID 有扰动时的飞行实验结果

#### 1.4.2.2 优先级控制分配对比伪逆

本实验用于验证设计的优先级控制分配算法在实际飞行过程中的表现情况。实验条件的设计与仿真实验中保持一致，扰动的添加方式采用图1-3b中的设计，三轴期望姿态都被设置为0。对照实验采用伪逆法，两种分配算法的执行步长均为0.01s。

姿态通道的实际飞行实验结果如图1-11所示。三轴的姿态误差变化情况与主动扰动产生的力矩变化情况相一致。但是在优先级控制分配算法下，三轴姿态通道的姿态误差幅值更小，并且误差收敛的速度也更快。上述结果与仿真实验中保持一致，进一步说明了仿真实验中结果分析的正确性。

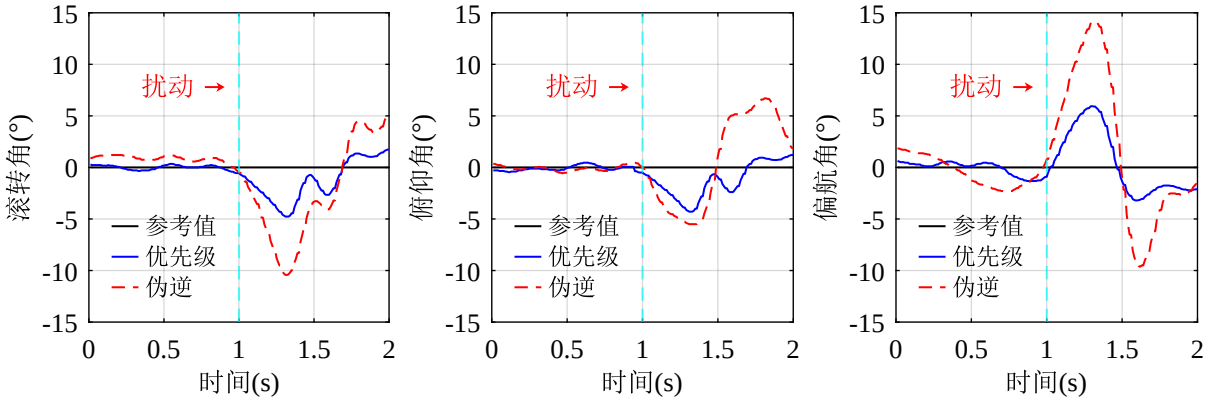


图 1-11 优先级对比伪逆的姿态保持飞行实验结果

两种分配方法的分配误差的比较如图1-12所示。分配误差于式(1-66)和式(1-67)中定义。从图中可以看出，伪逆方法下的总体分配误差远大于优先级控制分配算法下的分配误差，这导致了伪逆方法下的姿态误差也更大。这说明本文设计的优先级控制分配算法在实际飞行中同样能够提高姿态控制的精度。

实际飞行实验中优先级分量的分配误差比较如图1-13所示。从图中可以看出，即使

是在实际飞行中, INDI 输入分量  $\nu_i$  也始终满足可达性约束。反馈分配误差  $e_i$  的存在表明优先级控制分配算法对  $\nu_f$  进行了截断。通过实际飞行实验进一步验证了优先级控制分配算法的可行性和有效性。

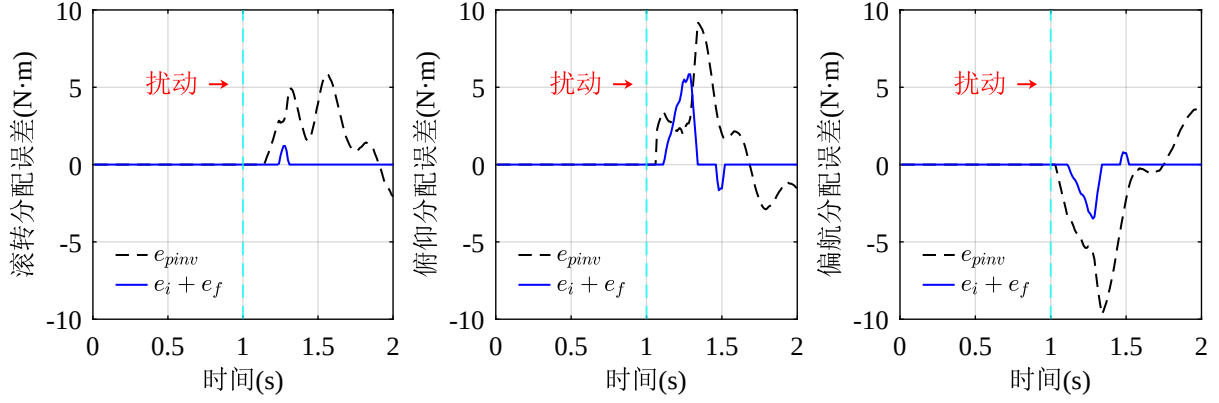


图 1-12 优先级对比伪逆的分配误差飞行实验结果

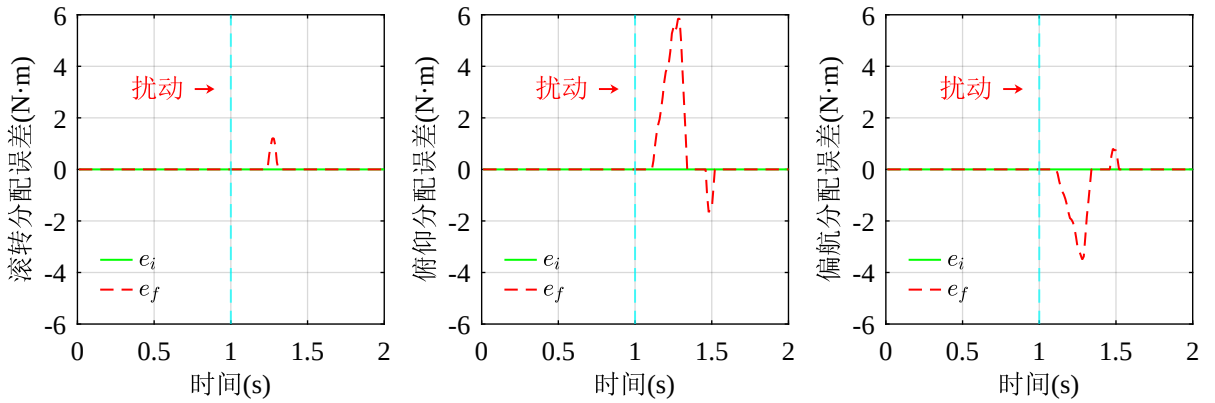


图 1-13 优先级各分量的分配误差飞行实验结果

## 1.5 本章小结

本章围绕 DFUAV 的姿态控制问题展开研究, 系统性地构建了基于 INDI 与优先级控制分配的理论框架及工程实现方案。首先介绍了 INDI 方法和控制分配理论以及两者在分层控制架构中的作用, 然后通过姿态动力学模型简化降低了姿态动力学的非线性复杂度。进一步, 提出了一种融合 INDI 架构与优先级控制分配的姿态控制策略, 基于传感器的 INDI 方法借助陀螺仪对难以测得的力矩有较好的补偿能力, 基于优先级控制分配可以最大程度保障虚拟控制输入中高优先级分量的可达性。最后通过一系列 MATLAB 仿真与实际飞行实验, 证实了所提出的姿态控制方案相比于 PID 控制在扰动条件下仍能保持较小的跟踪误差, 且控制分配误差也比传统的伪逆法更小。充分验证了所设计的姿态控制器的有效性, 为下文的位置控制器设计奠定了基础。

## 第二章 涵道风扇式无人机的位置控制

为了实现对轨迹规划结果的准确跟踪，一个跟踪性能优良、抗扰能力强的位置控制器是必不可少的。本文构建了基于分层解耦的外环-内环模块化控制框架，外环控制即位置控制器，内环控制即姿态控制器。位置控制作为轨迹跟踪的核心决策单元，首先通过融合 IMU、卫星定位模块和气压计等多源传感器数据，使用基于误差状态的卡尔曼滤波 (ESKF) 算法实时解算无人机在三维空间中的位置、速度等系统状态信息。然后根据当前时刻的位置误差、速度误差和受力状态因素，计算出涵道风扇的总拉力 (对应转速指令  $\Omega$ )，并结合期望的姿态指令  $\eta_d = [\varphi_d \ \theta_d \ \psi_d]^T$ ，实现期望的位置、速度控制。基于解耦的模块化控制框架，实际的位置跟踪问题可以按照单独的外环控制来考虑，而不用考虑内环控制。并且模块化设计使得位置控制回路可以独立于姿态动力学环节进行控制器参数调节，有效地降低了 DFUAV 这一多变量耦合系统的设计复杂度。对于内环控制，则采用第三章介绍的基于 INDI 和优先级控制分配的姿态控制策略，期望的姿态指令由位置控制器解算得到。

在户外复杂气流环境中，阵风引起的非定常气流扰动是制约 DFUAV 位置控制精度的重要因素。计算流体力学仿真研究表明，当环境风速达到 4.6m/s 时，建筑物周围空气流速最高可以达到 7.6m/s，这无疑会对 DFUAV 造成非常强烈的干扰，显著影响 DFUAV 的稳定性。因此，设计一个能够有效抗风的位置控制器尤为重要。针对该问题，广泛使用的 PID 位置控制器在有阵风的情况下存在其固有的缺陷：积分环节 I 在面对持续的风扰动时，补偿效果具有明显的相位滞后特性；而微分环节面对高频风速干扰则容易引发超调和震荡。如果环境风速变化已知，那么就可以及时补偿扰动以减小位置误差。为了测量环境风速，一种可能的解决方案是使用风速计测量机体速度与环境风速的相对速度，但环境风速可能来自各个方向，所以至少需要在无人机机体上安装三个风速计，这无疑会增加 DFUAV 硬件系统设计的复杂性和成本。并且风速计在低速的情况下存在明显噪声，误差较大。

第三章介绍了基于传感器的 INDI 方法的优越性：使用陀螺仪的测量值进行一阶差分来计算角加速度，进而基于角加速度的测量值估计出一个控制周期内对未建模动态力矩的影响并实时补偿。牛顿第二定律揭示了加速度的大小与力的大小成正比关系 ( $\Delta \mathbf{F} \propto \Delta \mathbf{a}$ )，所以基于机载 IMU 中的加速度计三轴比力的测量值，可以直接估计出 DFUAV 所受环境风扰动的等效外力矢量。这一点与使用陀螺仪估计未建模动态力矩变

化的思路是一致的。既然 DFUAV 的姿态可以被控制，那么位置控制同样可以采用这种思路，设计出一个用于控制 DFUAV 线性加速度的增量控制器。

因此本章安排如下：首先对位置动力学模型进行简化处理以适应 INDI 位置控制器的设计。然后根据加速度计可估算力这一特点，设计基于 INDI 的位置控制器。最后进行仿真实验与实际飞行实验，验证所提出的位置控制器的有效性。

## 2.1 位置动力学模型简化

在设计位置控制器之前，首先需要对位置动力学模型进行简化处理。根据本章开篇时的分析，可以使用 IMU 中的加速度计对无人机机体加速度的测量来估计算机体所受外力。加速度计对加速度的测量本质上是基于比力的物理概念。在惯性导航系统中，比力被定义为无人机所受的合外力 (包括气动力、推进力等) 与重力矢量作向量差然后再除以飞行器的总质量。比力具有与加速度相同的量纲 ( $\text{m/s}^2$ )。根据式(??)， $\mathbf{F}^b$  表示的合外力已除去重力作用，所以比力在机体坐标系下可以表示为：

$$\mathbf{A}^b = \begin{bmatrix} A_x^b \\ A_y^b \\ A_z^b \end{bmatrix} = \frac{1}{m} \mathbf{F}^b \quad (2-1)$$

DFUAV 位置子系统的控制输入为涵道风扇的转速  $\Omega$ ，为与加速度量纲统一，类似姿态控制，结合式(??)引入具有加速度量纲的位置控制的虚拟控制输入  $\mathbf{a}_u$ ：

$$\begin{aligned} \mathbf{a}_u &= \frac{1}{m} \mathbf{R}_b^e \mathbf{F}_{fan}^b \\ &= \frac{1}{m} \mathbf{R}_b^e \begin{bmatrix} 0 \\ 0 \\ -k_{fan} \Omega^2 \end{bmatrix} \end{aligned} \quad (2-2)$$

因此只要求解出期望的  $\mathbf{a}_u$  即可根据无人机质量、旋转矩阵 (姿态) 和系数  $k_{fan}$  得到期望的涵道风扇转速  $\Omega$ 。

由于位置平移运动在地面坐标系下分析较为方便，因此式(2-2)中左乘以旋转矩阵  $\mathbf{R}_b^e$ ，可以在地面坐标系下表示三轴加速度输入。其中  $\mathbf{a}_u$  可描述为无人机姿态  $\boldsymbol{\eta}$  和风扇转速  $\Omega$  的函数：

$$\mathbf{a}_u = \mathbf{a}_u(\boldsymbol{\eta}, \Omega) \quad (2-3)$$

类似地，将不包括重力作用的合外力矢量  $\mathbf{F}^b$  除去涵道风扇的拉力矢量统一表示为 DFUAV 的气动力，使用  $\bar{\mathbf{A}}^e$  表示气动力在地面坐标系下的加速度：

$$\bar{\mathbf{A}}^e = \begin{bmatrix} A_x^e \\ A_y^e \\ \bar{A}_z^e \end{bmatrix} = \mathbf{R}_b^e \begin{bmatrix} A_x^b \\ A_y^b \\ \bar{A}_z^b \end{bmatrix} = \frac{1}{m} \mathbf{R}_b^e (\mathbf{F}^b - \mathbf{F}_{fan}^b) \quad (2-4)$$

根据建模分析中  $\mathbf{F}^b$  的组成可知， $\bar{\mathbf{A}}^e$  可表示为姿态  $\boldsymbol{\eta}$ 、速度  $\mathbf{V}^e$  和环境风速  $\mathbf{W}^e$  的函数：

$$\bar{\mathbf{A}}^e = \bar{\mathbf{A}}^e(\boldsymbol{\eta}, \mathbf{V}^e, \mathbf{W}^e) \quad (2-5)$$

结合式(??)、(2-3)和(2-5)，可以得到位置动力学模型的简化形式：

$$\begin{aligned} \dot{\mathbf{V}}^e &= \frac{1}{m} \mathbf{R}_b^e \mathbf{F}^b + g \mathbf{e}_3 \\ &= \bar{\mathbf{A}}^e + \mathbf{a}_u + g \mathbf{e}_3 \\ &= \begin{bmatrix} A_x^e \\ A_y^e \\ \bar{A}_z^e \end{bmatrix} + \begin{bmatrix} a_{ux} \\ a_{uy} \\ a_{uz} \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ g \end{bmatrix} \end{aligned} \quad (2-6)$$

根据简化后的位置动力学模型(2-6)，可将动力学分量分为两部分：第一部分为外环虚拟控制输入  $\mathbf{a}_u$ ，这部分分量与实际的外环控制输入  $\Omega$  有关，用于控制外环状态 (无人机的位置、速度)，将通过下一小节的 INDI 位置控制器设计求解；第二部分为  $\bar{\mathbf{A}}^e + g \mathbf{e}_3$ ，可以认为这部分分量是机体受到的扰动，会导致位置误差，将通过 INDI 位置控制器进行扰动补偿。

## 2.2 基于 INDI 的位置控制器设计

本节将 INDI 控制方法进一步应用于位置 (外环) 控制中，使用机载 IMU 中的加速度计来估计环境风速扰动，并通过 INDI 位置控制器根据扰动的大小实时补偿以提高 DFUAV 的抗风性能。这种外环-内环协同的扰动观测补偿机制具有显著的技术优势：一方面，加速度计对外力变化敏感，能够以 1000Hz 的频率检测环境风速扰动，相比于本章开篇时提到的风速计方案避免了安装复杂度和低速情况下的测量噪声问题；另一方面，内外环均采用基于传感器的扰动补偿策略，形成了从力矩扰动到力扰动的全链路扰动观测与补偿闭环控制框架，在阵风场景下可以抵消姿态震荡与位置偏移的耦合效应，

可以有效地提高系统的稳定性和鲁棒性。

### 2.2.1 INDI 位置控制架构

不同于姿态旋转运动(??)，位置平移运动(??)不涉及三个位置通道的耦合，因此常采用如下的线性化位置误差：

$$\begin{aligned} \mathbf{e}_P &= \mathbf{P}_d^e - \mathbf{P}^e \\ &= \begin{bmatrix} x_d^e \\ y_d^e \\ z_d^e \end{bmatrix} - \begin{bmatrix} x^e \\ y^e \\ z^e \end{bmatrix} \end{aligned} \quad (2-7)$$

因此期望的速度指令可以表示为：

$$\mathbf{V}_d^e = \mathbf{K}_P \mathbf{e}_P + \frac{d\mathbf{P}_d^e}{dt} \quad (2-8)$$

此处采用比例控制加前馈控制的方式，其中  $\mathbf{K}_P = \text{diag}(K_{Px}, K_{Py}, K_{Pz})$  为位置误差的正定增益矩阵。

对于与动力学相关的速度动态，位置动力学模型简化中已经将其简化为以下形式：

$$\dot{\mathbf{V}}^e = \bar{\mathbf{A}}^e(\boldsymbol{\eta}, \mathbf{V}^e, \mathbf{W}^e) + \mathbf{a}_u + g\mathbf{e}_3 \quad (2-9)$$

可以直接在式(2-9)的基础上应用 INDI 方法。根据第三章的 INDI 相关理论，为了得到  $\dot{\mathbf{V}}^e$  的增量表达式，对式(2-9)在工作点处作一阶泰勒展开：

$$\begin{aligned} \dot{\mathbf{V}}^e &= \bar{\mathbf{A}}^e(\boldsymbol{\eta}_0, \mathbf{V}_0^e, \mathbf{W}_0^e) + \mathbf{a}_{u0} + g\mathbf{e}_3 \\ &+ \left. \frac{\partial \bar{\mathbf{A}}^e}{\partial \boldsymbol{\eta}} \right|_{\boldsymbol{\eta}=\boldsymbol{\eta}_0} (\boldsymbol{\eta} - \boldsymbol{\eta}_0) + \left. \frac{\partial \bar{\mathbf{A}}^e}{\partial \mathbf{V}^e} \right|_{\mathbf{V}^e=\mathbf{V}_0^e} (\mathbf{V}^e - \mathbf{V}_0^e) \\ &+ \left. \frac{\partial \bar{\mathbf{A}}^e}{\partial \mathbf{W}^e} \right|_{\mathbf{W}^e=\mathbf{W}_0^e} (\mathbf{W}^e - \mathbf{W}_0^e) + (\mathbf{a}_u - \mathbf{a}_{u0}) \end{aligned} \quad (2-10)$$

在位置动力学模型简化小节中，定义了具有加速度量纲的  $\mathbf{a}_u(\boldsymbol{\eta}, \Omega)$ ，用来表示位置控制的虚拟控制输入，所以泰勒展开式中不对其细化求偏导数，而是将其看作一个整体来进行求解。

展开式中的第一项  $\bar{\mathbf{A}}^e(\boldsymbol{\eta}_0, \mathbf{V}_0^e, \mathbf{W}_0^e) + \mathbf{a}_{u0} + g\mathbf{e}_3$  等价于工作点处的加速度，这个值可以由机载 IMU 测量之后再转换到地面坐标系下并加上重力矢量得到，可以使用  $\dot{\mathbf{V}}_0^e$

表示：

$$\dot{\mathbf{V}}_0^e = \bar{\mathbf{A}}^e(\boldsymbol{\eta}_0, \mathbf{V}_0^e, \mathbf{W}_0^e) + \mathbf{a}_{u0} + g\mathbf{e}_3 \quad (2-11)$$

展开式中的其他项是关于系统内部状态变量  $(\mathbf{V}^e, \boldsymbol{\eta})$ 、系统控制输入  $\mathbf{a}_u$  和外部环境风速扰动  $\mathbf{W}^e$  的一阶偏导数项。根据第三章的 INDI 理论介绍部分的时间尺度分离法则，可以认为在加速度的输入信号  $\Omega$  (对应虚拟控制输入  $\mathbf{a}_u$ ) 的时间尺度内，系统内部的状态变量的变化是缓慢的，包括展开式中的速度  $\mathbf{V}^e$  和姿态  $\boldsymbol{\eta}$ 。因此可以将这些状态变量的一阶偏导数项近似视为 0：

$$\begin{cases} \mathbf{V}^e - \mathbf{V}_0^e \approx 0 \\ \boldsymbol{\eta} - \boldsymbol{\eta}_0 \approx 0 \end{cases} \quad (2-12)$$

此外，对于外部环境的风速扰动  $\mathbf{W}^e$ ，类似姿态控制方案设计中的分析，最佳的假设同样是认为  $(\mathbf{W}^e - \mathbf{W}_0^e \approx 0)$ 。所有由风速变化导致的空气动力作用都已经包含在了  $\dot{\mathbf{V}}_0^e$  中。

其余关于系统控制输入项为  $(\mathbf{a}_u - \mathbf{a}_{u0})$ 。因此可以得到速度动态的增量表达式：

$$\begin{aligned} \dot{\mathbf{V}}^e &= \dot{\mathbf{V}}_0^e + \mathbf{a}_u - \mathbf{a}_{u0} \\ &= (\mathbf{R}_b^e)_0 \begin{bmatrix} A_{x0}^e \\ A_{y0}^e \\ A_{z0}^e \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ g \end{bmatrix} + \begin{bmatrix} a_{ux} \\ a_{uy} \\ a_{uz} \end{bmatrix} - \begin{bmatrix} a_{ux0} \\ a_{uy0} \\ a_{uz0} \end{bmatrix} \end{aligned} \quad (2-13)$$

INDI 通过对式(2-13)求逆，得到期望的位置控制虚拟控制输入的增量表达式：

$$\begin{aligned} \mathbf{a}_{ud} &= \boldsymbol{\kappa} - \dot{\mathbf{V}}_0^e + \mathbf{a}_{u0} \\ &= \begin{bmatrix} \boldsymbol{\kappa}_x \\ \boldsymbol{\kappa}_y \\ \boldsymbol{\kappa}_z \end{bmatrix} - (\mathbf{R}_b^e)_0 \begin{bmatrix} A_{x0}^e \\ A_{y0}^e \\ A_{z0}^e \end{bmatrix} - \begin{bmatrix} 0 \\ 0 \\ g \end{bmatrix} + \begin{bmatrix} a_{ux0} \\ a_{uy0} \\ a_{uz0} \end{bmatrix} \end{aligned} \quad (2-14)$$

其中，使用  $\mathbf{a}_{ud}$  替换  $\mathbf{a}_u$  表明是期望的虚拟控制输入。

将式(2-14)代入式(2-13)所示的增量表达式中，可以得到线性化的速度动态方程：

$$\dot{\mathbf{V}}^e = \boldsymbol{\kappa} \quad (2-15)$$

对于式(2-15)所示的一阶积分系统，仅使用具有正反馈增益的比例控制器就可以确

保系统的稳定性，为提高系统响应速度和精度，此处也可以加入前馈控制：

$$\kappa = \mathbf{K}_V(\mathbf{V}_d^e - \mathbf{V}^e) + \frac{d\mathbf{V}_d^e}{dt} \quad (2-16)$$

值得一提的是，前馈控制的加入不会影响系统传递函数的分母，即加入前后系统的极点不变，从而稳定性也不受影响。

将式(2-16)代入式(2-15)中可以得到：

$$\dot{\mathbf{V}}^e = \mathbf{K}_V(\mathbf{V}_d^e - \mathbf{V}^e) + \frac{d\mathbf{V}_d^e}{dt} \quad (2-17)$$

其中  $\mathbf{K}_V = \text{diag}(K_{Vx}, K_{Vy}, K_{Vz})$  为正反馈增益矩阵。

将式(2-16)代入式(2-14)中可以得到：

$$\begin{aligned} \mathbf{a}_{ud} &= \mathbf{K}_V(\mathbf{V}_d^e - \mathbf{V}^e) + \frac{d\mathbf{V}_d^e}{dt} - \dot{\mathbf{V}}_0^e + \mathbf{a}_{u0} \\ &= \begin{bmatrix} K_{Vx} & & \\ & K_{Vy} & \\ & & K_{Vz} \end{bmatrix} \left( \begin{bmatrix} V_{dx}^e \\ V_{dy}^e \\ V_{dz}^e \end{bmatrix} - \begin{bmatrix} V_x^e \\ V_y^e \\ V_z^e \end{bmatrix} \right) + \begin{bmatrix} \frac{dV_{dx}^e}{dt} \\ \frac{dV_{dy}^e}{dt} \\ \frac{dV_{dz}^e}{dt} \end{bmatrix} - (\mathbf{R}_b^e)_0 \begin{bmatrix} A_{x0}^e \\ A_{y0}^e \\ A_{z0}^e \end{bmatrix} - \begin{bmatrix} 0 \\ 0 \\ g \end{bmatrix} + \begin{bmatrix} a_{ux0} \\ a_{uy0} \\ a_{uz0} \end{bmatrix} \end{aligned} \quad (2-18)$$

文献[7] 中提供了另一种思路来寻找所需加速度输入的增量，该思路基于非线性方法。由于与动力学输入相关的非线性速度动态方程(2-9)是已知的，因此可以进行非线性反演。由于没有关于  $\bar{\mathbf{A}}^e(\boldsymbol{\eta}, \mathbf{V}^e, \mathbf{W}^e)$  的精确估计，所以保持增量结构仍然是适合的。对方程(2-9)两边同时减去同一公式的此前最近一个工作点的表达式可以得到：

$$\dot{\mathbf{V}}^e - \dot{\mathbf{V}}_0^e = \bar{\mathbf{A}}^e(\boldsymbol{\eta}, \mathbf{V}^e, \mathbf{W}^e) - \bar{\mathbf{A}}^e(\boldsymbol{\eta}_0, \mathbf{V}_0^e, \mathbf{W}_0^e) + \mathbf{a}_u - \mathbf{a}_{u0} + g\mathbf{e}_3 - g_0\mathbf{e}_3 \quad (2-19)$$

假设在这个小的时间瞬间内， $\bar{\mathbf{A}}^e(\boldsymbol{\eta}, \mathbf{V}^e, \mathbf{W}^e)$  和  $g$  的变化很小，所以有：

$$\dot{\mathbf{V}}^e - \dot{\mathbf{V}}_0^e = \mathbf{a}_u - \mathbf{a}_{u0} \quad (2-20)$$

这个简化方程将虚拟控制输入的增量与加速度的增量联系起来，与式(2-13)的结果是一致的。

类似姿态控制，位置控制中的 INDI 在每一个工作点计算的虚拟控制输入  $\mathbf{a}_{ud}$  都是基于上一个工作点处的虚拟控制输入  $\mathbf{a}_u$  加上期望的加速度  $\dot{\mathbf{V}}^e$  再减去此刻通过加速度计测量值转换得到的加速度  $\dot{\mathbf{V}}_0^e$ 。而  $\dot{\mathbf{V}}_0^e$  中已经包含了相邻两个工作点之间气动力对机体的影响，因此负面影响不会随着时间积累，而是在每一个工作点实时补偿，有效地提

升了系统的抗风扰性能。

由于  $\dot{\mathbf{V}}_0^e$  的获取依赖于加速度计的测量值，而加速度计又对机体震动十分敏感，在实际应用中的常用的做法是在飞控板所处的上部电子仓与机身连接处安装减震垫以减小机体震动对加速度计的影响。此外，加速度计的测量值还需要进行滤波处理。第三章介绍的姿态控制器中用到的陀螺仪也需要滤波处理，假设内环使用滤波器  $f_{in}$  过滤，内环的推力增量为  $\mathbf{T} - \mathbf{T}_{f_{in}}$ ，外环使用滤波器  $f_{out}$  过滤，外环的推力增量为  $\mathbf{T} - \mathbf{T}_{f_{out}}$ 。为了使外环的推力增量可以传递到内环，两个滤波器的形式和参数都需要相同<sup>[7]</sup>：

$$f_{in} = f_{out} \quad (2-21)$$

所以加速度计使用的滤波器也是二阶巴特沃斯低通滤波器，截止频率为 30Hz。

此外，为了保证所有下标为 0 的项都位于同一工作点，实践中还需要对虚拟控制输入和姿态进行同步滤波。为与内环滤波统一，同样使用下标  $(.)_s$  表示经过滤波器滤波后的项。

$$\mathbf{a}_{ud} = \mathbf{K}_V(\mathbf{V}_d^e - \mathbf{V}^e) + \frac{d\mathbf{V}_d^e}{dt} - \dot{\mathbf{V}}_f^e + \mathbf{a}_{uf} \quad (2-22)$$

除了滤波外，加速度计测量的原始值往往有固定偏置，加速度计偏置的存在意味着 DFUAV 在静止状态下会有一个非零的加速度测量值，这种系统性误差会导致惯性导航结算中的速度与位置出现积分漂移，增加误差。为了消除这个偏置，需要在使用之前对加速度计进行校准。一般情况下采用六面校准法即可，若对精度要求高，则可采用高精度转台进行动态校准。

### 2.2.2 期望姿态角与拉力解算

期望的加速度输入  $\mathbf{a}_{ud}$  是在地面坐标系下的，而 DFUAV 的拉力向量始终作用在机体坐标系的  $\mathbf{O}_b - \mathbf{Z}_b$  轴方向上。根据式(2-2)以及旋转矩阵不影响矢量大小的性质，可以得到期望的涵道风扇拉力大小  $\|\mathbf{F}_{fan}\|$ ：

$$\begin{aligned} \|\mathbf{F}_{fan}\|_2 &= m\|\mathbf{a}_{ud}\|_2 \\ &= m\sqrt{a_{udx}^2 + a_{udy}^2 + a_{udz}^2} \end{aligned} \quad (2-23)$$

根据拉力大小可求出期望的涵道风扇的转速  $\Omega_d$ ：

$$\Omega_d = \sqrt{\frac{\|\mathbf{F}_{fan}\|_2}{k_{fan}}} \quad (2-24)$$

由于偏航角  $\psi$  对 DFUAV 的飞行轨迹不产生影响，所以在位置控制时可以人为自主设定期望的偏航角。一般情况下设定期望的偏航角为实际运动方向与地理北极的夹角，即期望的轨迹方向。例如在轨迹跟踪过程中，假设当前时刻的参考位置为  $\mathbf{P}_d^e = [x_d^e \ y_d^e \ z_d^e]^T$ ，上一时刻的参考位置为  $\mathbf{P}_{d-1}^e = [x_{d-1}^e \ y_{d-1}^e \ z_{d-1}^e]^T$ ，则期望的偏航角  $\psi_d$  可以设置为：

$$\psi_d = \text{atan2}(y_d^e - y_{d-1}^e, x_d^e - x_{d-1}^e) \quad (2-25)$$

其中  $\text{atan2}$  表示接收两个参数的反正切函数，返回的角度范围是  $(-\pi, \pi]$ 。

为了求解期望的滚转角  $\varphi_d$  和俯仰角  $\theta_d$ ，本文采用几何解算方法，首先计算出期望的姿态旋转矩阵，再根据旋转矩阵求解出期望的滚转角和俯仰角。设期望的姿态旋转矩阵为  $\mathbf{R}_d$ ，则有：

$$\mathbf{R}_d = \begin{bmatrix} b_1 & b_2 & b_3 \end{bmatrix} \quad (2-26)$$

其中， $b_1$ 、 $b_2$  和  $b_3$  分别为期望的姿态旋转矩阵  $\mathbf{R}_d$  的三个列向量。分别表示机体坐标系的三个轴在地面坐标系下的方向向量。

由于虚拟控制输入的矢量和方向与机体坐标系  $\mathbf{O}_b - \mathbf{Z}_b$  轴方向相同，所以有：

$$b_3 = \frac{\mathbf{a}_{ud}}{\|\mathbf{a}_{ud}\|_2} \quad (2-27)$$

为了计算出  $b_1$  和  $b_2$ ，引入中间坐标系  $c$  系，如图2-1所示。中间坐标系的  $\mathbf{O}_c - \mathbf{Z}_c$  轴与地面坐标系的  $\mathbf{O}_e - \mathbf{Z}_e$  轴重合，中间坐标系的  $\mathbf{O}_c - \mathbf{X}_c$  轴和  $\mathbf{O}_c - \mathbf{Y}_c$  轴分别与地面坐标系的  $\mathbf{O}_e - \mathbf{X}_e$  轴和  $\mathbf{O}_e - \mathbf{Y}_e$  轴成  $\psi_d$  的夹角。因此  $\mathbf{O}_c - \mathbf{X}_c$  轴在地面坐标系下的方向向量为：

$$c_1 = \begin{bmatrix} \cos(\psi_d) \\ \sin(\psi_d) \\ 0 \end{bmatrix} \quad (2-28)$$

显然，中间坐标系的  $\mathbf{O}_c - \mathbf{X}_c$  轴处于机体坐标系的  $\mathbf{O}_b - \mathbf{X}_b - \mathbf{Z}_b$  平面内，因此  $\mathbf{O}_c - \mathbf{X}_c$  轴垂直于  $\mathbf{O}_b - \mathbf{Y}_b$  轴，所以  $b_2$  在地面坐标系下的方向向量为：

$$b_2 = \frac{b_3 \times c_1}{\|b_3 \times c_1\|_2} \quad (2-29)$$

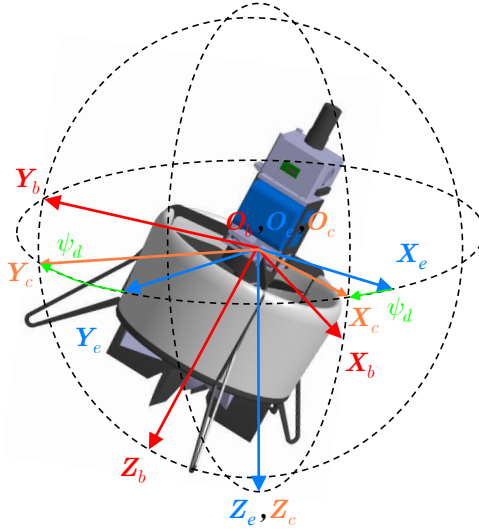


图 2-1 中间坐标系示意图

式(2-29)中如果分母为 0，那么表明  $b_3$  和  $c_1$  共线，这种情况下有俯仰角  $\theta_d = \pm 90^\circ$ 。对于本文研究的 DFUAV，由于实际姿态指令和控制舵面角度的约束，不会存在这种情况。

最后，根据右手定则求解出  $b_1$ ：

$$b_1 = b_2 \times b_3 \quad (2-30)$$

设  $R_{i,j}$  表示期望旋转矩阵  $R_d$  的第  $i$  行第  $j$  列元素，根据第二章式(??)定义的‘Z-Y-X’旋转顺序的旋转矩阵，可以求得期望的滚转角  $\varphi_d$  和俯仰角  $\theta_d$ ：

$$\begin{cases} \varphi_d = \tan^{-1} \left( \frac{\sin \varphi}{\cos \varphi} \right) = \tan^{-1} \left( \frac{R_{32}}{R_{33}} \right) \\ \theta_d = \sin^{-1} (\sin \theta) = \sin^{-1} (-R_{31}) \end{cases} \quad (2-31)$$

完整的基于 INDI 的位置控制系统框图如图2-2所示。

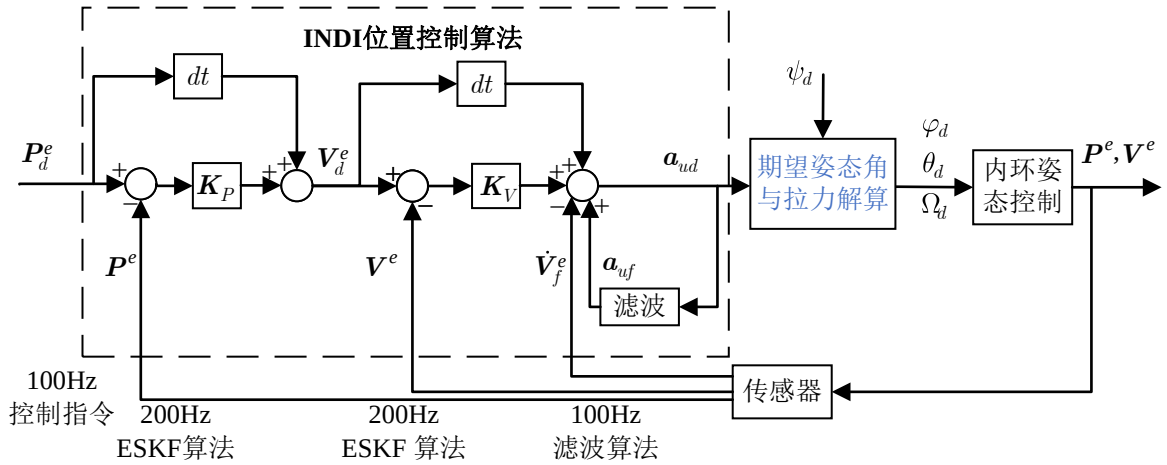


图 2-2 INDI 位置控制系统框图

## 2.3 实验结果和分析

为了检验本章提出的基于 INDI 的位置控制器设计的有效性, 本节进行了一系列仿真实验与实际飞行实验。仿真实验包括验证 INDI 位置控制器在外部环境风速扰动下的性能, 以及 INDI 位置控制器的轨迹跟踪性能, 仿真实验均使用 PID 位置控制器作为比较。在串级 PID 位置控制方法中, 位置环和速度环均采用了 PI 控制器。实际飞行实验中使用 INDI 位置控制器跟踪方形轨迹和圆形轨迹以验证控制器的轨迹跟踪能力。实验过程中使用的控制器参数如表2-1所示。仿真实验平台和实际飞行实验平台均与第三章的姿态控制实验相同, 为了进行可变控制, 在验证位置控制器性能时, 内环的姿态控制器均采用第三章设计的基于 INDI 和优先级控制分配的姿态控制器。

表 2-1 INDI 位置控制器和 PID 位置控制器的控制参数

| INDI 控制器参数符号 | 数值  | PID 控制器参数符号 | 数值  | PID 控制器参数符号 | 数值   |
|--------------|-----|-------------|-----|-------------|------|
| $K_{Px}$     | 0.5 | $K_{Px}$    | 0.5 | $I_{Px}$    | 1    |
| $K_{Py}$     | 0.5 | $K_{Py}$    | 0.5 | $I_{Py}$    | 1    |
| $K_{Pz}$     | 2.0 | $K_{Pz}$    | 2.0 | $I_{Pz}$    | 0.05 |
| $K_{Vx}$     | 1.5 | $K_{Vx}$    | 1.5 | $I_{Vx}$    | 0.25 |
| $K_{Vy}$     | 1.5 | $K_{Vy}$    | 1.5 | $I_{Vy}$    | 0.25 |
| $K_{Vz}$     | 2.0 | $K_{Vz}$    | 2.0 | $I_{Vz}$    | 0    |

### 2.3.1 仿真实验

#### 2.3.1.1 悬停抗风实验

本实验用于验证在 MATLAB 仿真环境中搭建的 INDI 位置控制器在模拟环境风速扰动情况下的定点悬停性能, 采用 PID 位置控制器作为对比。模拟的环境风速扰动的数学表达式如下:

$$\|\mathbf{W}^e\|_2 = V_{mean} + V_{low\_freq} + V_{high\_freq} + V_{noise}$$

$$\text{其中} \begin{cases} V_{mean} = 5 \\ V_{low\_freq} = \sin(2\pi f_{low\_freq} t) \\ V_{high\_freq} = 0.5 \sin(2\pi f_{high\_freq} t) \\ V_{noise} = 0.1 \text{rand}(t) \end{cases} \quad (2-32)$$

实验中采用四部分分量组成模拟环境风速扰动，分别是平均基本风速  $V_{mean}$ 、低频正弦风速  $V_{low\_freq}$  模拟大气尺度的缓慢波动、高频正弦风速  $V_{high\_freq}$  模拟小尺度湍流和随机噪声扰动  $V_{noise}$ 。其中低频正弦风速的频率为 0.05Hz，高频正弦风速的频率为 1Hz。风速随时间变化的波形如图2-3所示。

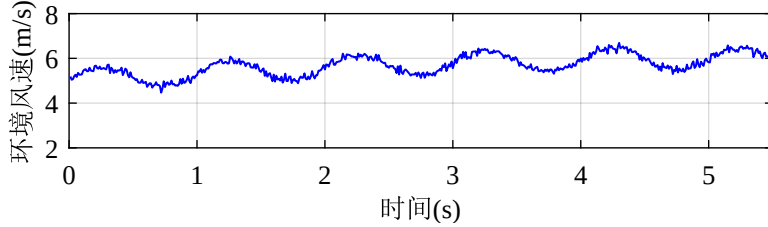


图 2-3 模拟环境风速扰动结果

为方便结果分析，风速扰动仅被添加到  $O_e - X_e$  方向上，所以有：

$$\mathbf{W}^e = \begin{bmatrix} \|\mathbf{W}^e\|_2 \\ 0 \\ 0 \end{bmatrix} \quad (2-33)$$

实验过程中期望的偏航角始终指向正北方向，即  $(\psi_d = 0)$ ，初始位置为  $\mathbf{P}_{start}^e = [0 \ 0 \ -10]^T \text{m}$ ，期望位置指令设置为  $\mathbf{P}_d^e = [0 \ 0 \ -10]^T \text{m}$ ，即保持定点，初始速度为  $\mathbf{V}_{start}^e = [0 \ 0 \ 0]^T \text{m/s}$ 。仿真时长为 10s，仿真步长为 0.01s，扰动在第 1s 时刻添加。图2-4展示了实验中分别使用 INDI 位置控制器和 PID 位置控制器的 DFUAV 的位置随时间变化的响应曲线。

比较两个实验中的位置响应曲线可知，基于所设定的来自 X 轴正向风速扰动，从扰动添加的第 1s 时刻开始，在两种控制器的控制下，DFUAV 在 X 轴的位置均发生了一定程度的偏移。但是 INDI 位置控制器的位置响应曲线在 X 轴的最大误差为 0.18m，而 PID 位置控制器的位置响应曲线在 X 轴的最大误差为 1.53m，远大于 INDI 控制下的位置误差。并且在第 4s 时刻，INDI 位置控制下 X 轴的位置误差已经近似收敛为 0，但是 PID 位置控制下的位置误差仍然较大且在仿真时间内无法收敛。这说明 INDI 位置控制器在外部环境风速扰动下的定点悬停性能优于 PID 位置控制器，性能优势归因于 INDI 位置控制器可以通过加速度计测量值估计外部环境的风速扰动并在每个控制周期内实时补偿，不会积累误差，从而提升了系统的抗风性能。而 PID 位置控制器保持位置稳定依赖于误差的积累，响应速度较慢，在外部风速扰动下无法实时补偿，导致位置误差无法收敛。

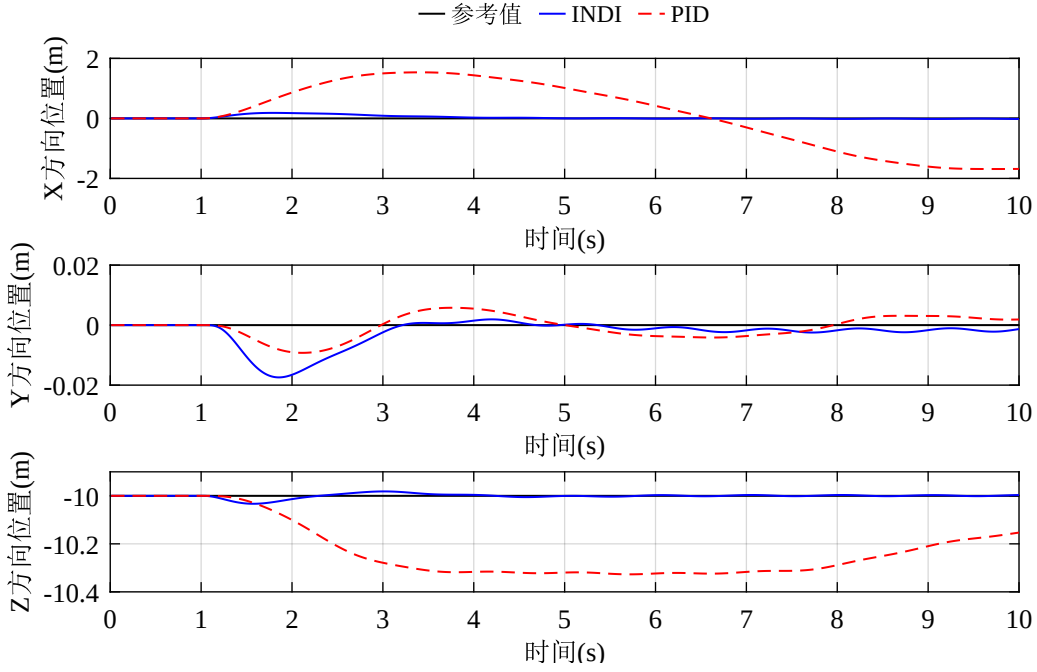


图 2-4 INDI 对比 PID 位置控制定点悬停抗风实验仿真结果

此外，从位置误差变化曲线的斜率来看，也能说明 PID 位置控制滞后性较强，而 INDI 位置控制器的响应速度更快。并且从图中可以看出，在 PID 位置控制下，X/Y 轴位置通道与 Z 轴高度通道会存在耦合，即在 X/Y 轴位置误差较大时，Z 轴高度误差也会增大，而 INDI 位置控制器的位置通道之间则没有这种耦合现象。

### 2.3.1.2 轨迹跟踪实验

本实验用于验证所提出的 INDI 位置控制器在执行轨迹跟踪任务下的性能。实验中选取了一个简单的斜坡轨迹作为参考轨迹，参考轨迹的数学表达式为：

$$\mathbf{P}_d^e(t) = \begin{bmatrix} x_d^e(t) \\ y_d^e(t) \\ z_d^e(t) \end{bmatrix} = \begin{bmatrix} t \\ 0 \\ -10 \end{bmatrix} \quad (2-34)$$

为了进行可变控制，轨迹跟踪实验中不添加外部环境风速扰动。实验过程中期望的偏航角始终指向正北方向，即  $(\psi_d = 0)$ ，初始位置为  $\mathbf{P}_{start}^e = [0 \ 0 \ -10]^T \text{m}$ ，初始速度为  $\mathbf{V}_{start}^e = [0 \ 0 \ 0]^T \text{m/s}$ ，仿真时长为 10s，仿真步长为 0.01s。图 2-5 展示了实验中分别使用 INDI 位置控制器和 PID 位置控制器的 DFUAV 的位置随时间变化的响应曲线。

根据两组实验的结果可以看出，无论是 INDI 位置控制器还是 PID 位置控制器，DFUAV 均能跟踪上期望的轨迹。但是 INDI 位置控制器的轨迹跟踪性能要明显优于 PID 位置控制器，一方面 INDI 位置控制器的轨迹跟踪响应曲线在 X 轴的最大跟踪误差为

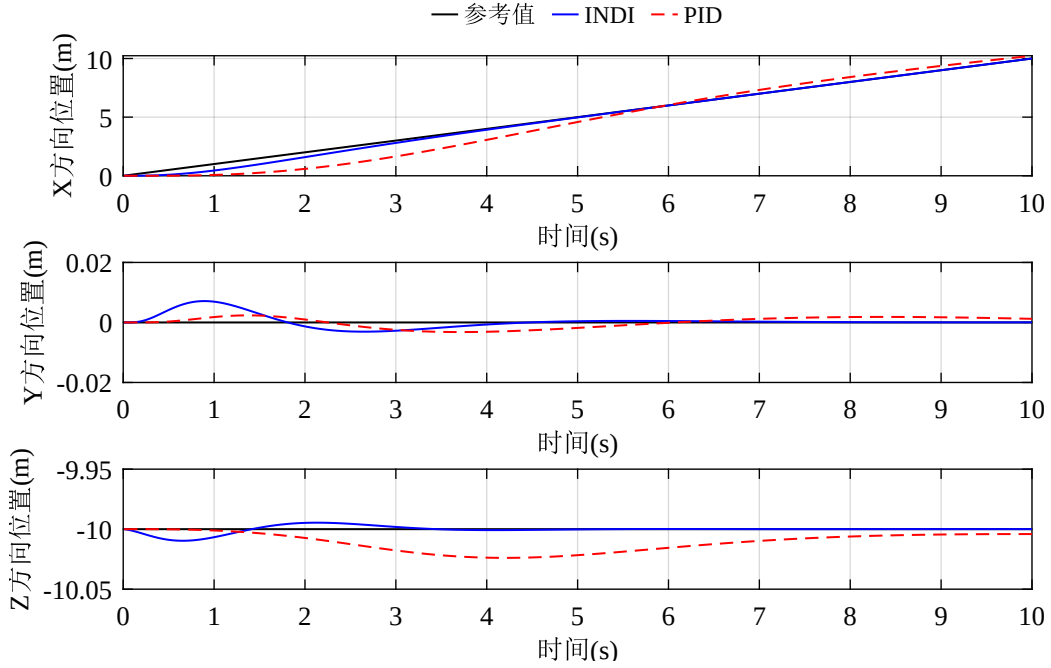


图 2-5 INDI 对比 PID 位置控制轨迹跟踪实验仿真结果

0.55m，而 PID 位置控制器的轨迹跟踪响应曲线在 X 轴的最大误差为 1.44m，远大于 INDI 控制下的位置误差。另一方面，INDI 控制下的位置曲线在第 4s 时就已基本无误差，但是在 PID 控制下的位置曲线在第 6s 时才跟踪上，并且由于积分的累计作用出现了超调，导致位置曲线在第 6s 后超过了期望轨迹。INDI 位置控制的轨迹跟踪性能优势主要得益于采用了位置环和速度环的前馈控制策略，提前根据输入的变化趋势进行控制，从而提升了系统的响应速度和精度。

通过上述两个仿真实验，证明了所设计的基于 INDI 的位置控制器具有较强的抗风性能和轨迹跟踪性能。

### 2.3.2 实际飞行实验

本小节通过飞行实验验证 INDI 位置控制器的轨迹跟踪性能。实验中选取方形轨迹和圆形轨迹作为参考轨迹以验证试验样机的点对点轨迹跟踪性能和圆弧轨迹跟踪性能。

#### 2.3.2.1 点对点轨迹飞行实验

点对点轨迹飞行实验的参考轨迹为  $40\text{m} \times 40\text{m}$  的方形，期望高度保持为  $-5\text{m}$ 。初始位置为  $\mathbf{P}_{start}^e = [0.48 \quad -14.28 \quad -5]^T \text{m}$ ，初始速度为  $\mathbf{V}_{start}^e = [0 \quad 0 \quad 0]^T \text{m/s}$ ，最大平飞速度设置为  $2\text{m/s}$ 。期望的偏航角始终指向期望轨迹的方向，在每处直角转弯点都会停下转向  $90^\circ$  然后继续跟踪期望轨迹。实验过程中，试验样机在地面坐标系下的位置跟踪曲线、速度跟踪曲线、姿态跟踪曲线和二维平面轨迹分别如图2-6到图2-9所示。

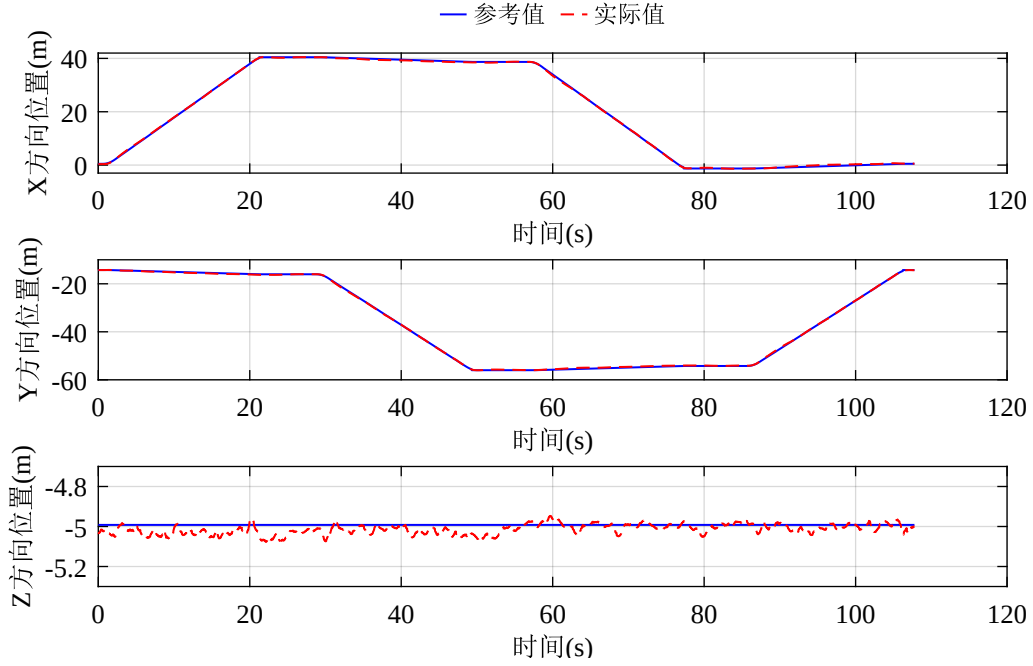


图 2-6 点对点轨迹飞行实验位置跟踪结果

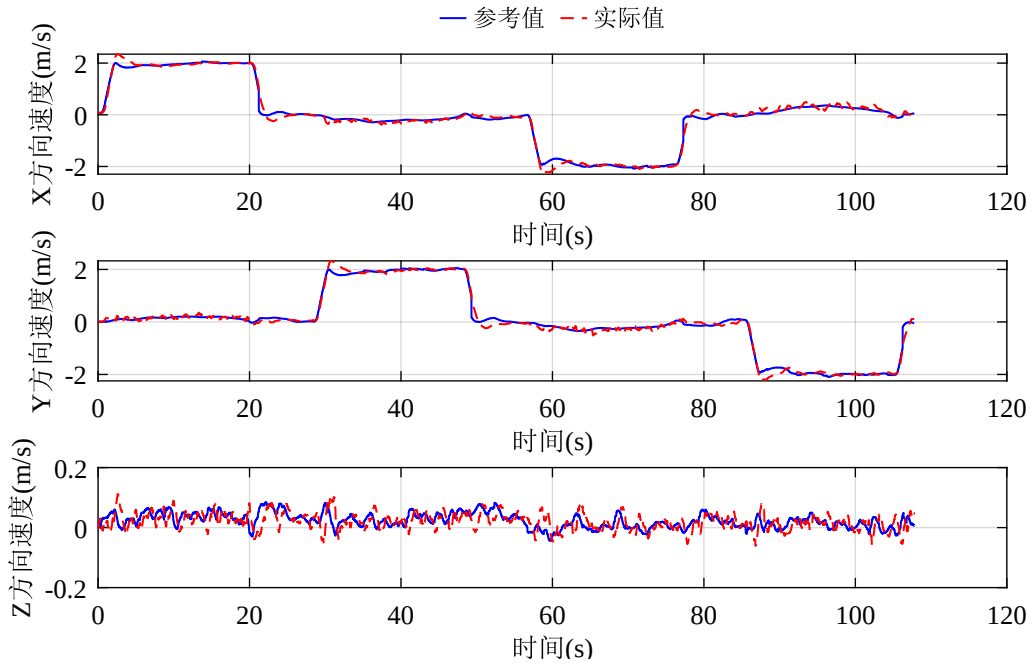


图 2-7 点对点轨迹飞行实验速度跟踪结果

根据实验结果可知，试验样机在轨迹跟踪过程中均能够保持在期望轨迹附近，飞行中三轴位置的最大误差为  $[0.47 \ 0.34 \ 0.08]\text{m}$ ，这是由于 X/Y 方向上的位置控制与期望的姿态角  $\varphi_d$  和  $\theta_d$  的跟踪精度有关，而高度通道的控制与姿态角无关，控制较为简单，因此高度误差相对较小。对于速度响应曲线，仅在每段加减速过程中有少许超调量，X 和 Y 方向的最大速度误差均为  $0.7\text{m/s}$ ，但是在加减速过程结束后，速度曲线均能够较好地稳定在期望速度上。由于飞行过程中期望偏航角  $\psi_d$  始终沿着飞行方向，因此 DFUAV

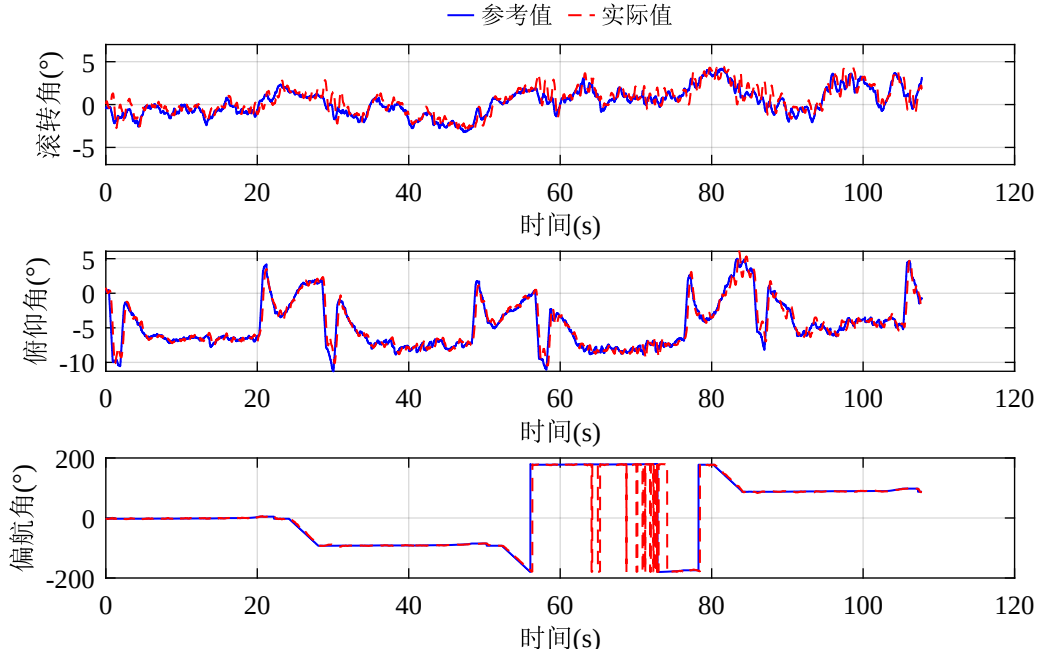


图 2-8 点对点轨迹飞行实验姿态跟踪结果

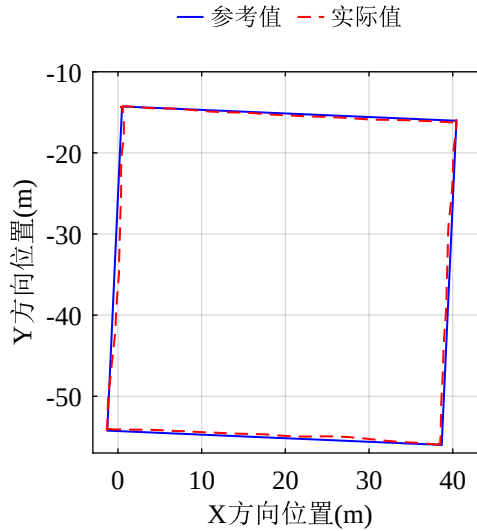


图 2-9 点对点轨迹飞行实验二维平面轨迹结果

的俯仰角会经历较大变化，根据姿态跟踪曲线可以看出，俯仰角的跟踪效果较好，最大误差为  $3.89^\circ$ ，无明显超调。滚转角和俯仰角的跟踪误差同样较小，位于合理范围内。

通过二维平面轨迹可以更加直观地看出，试验样机在整个轨迹飞行过程中能够较好地跟踪期望的方形轨迹，证明了点对点的轨迹跟踪性能较好。

### 2.3.2.2 圆形轨迹飞行实验

圆形轨迹飞行实验的参考轨迹为半径为  $25\text{m}$  的圆形，期望高度保持为  $-5\text{m}$ 。初始位置为  $\mathbf{P}_{start}^e = [18.98 \quad -15.14 \quad -5]^T \text{m}$ ，初始速度为  $\mathbf{V}_{start}^e = [0 \quad 0 \quad 0]^T \text{m/s}$ ，飞行的

最大线速度设置为  $2\text{m/s}$ 。期望的偏航角始终指向期望轨迹的方向，由式(2-25)计算得到，期望飞行的弧度为  $4\pi$ (两圈)。实验过程中，试验样机在地面坐标系下的位置跟踪曲线、速度跟踪曲线、姿态跟踪曲线和二维平面轨迹分别如图2-10到图2-13所示。

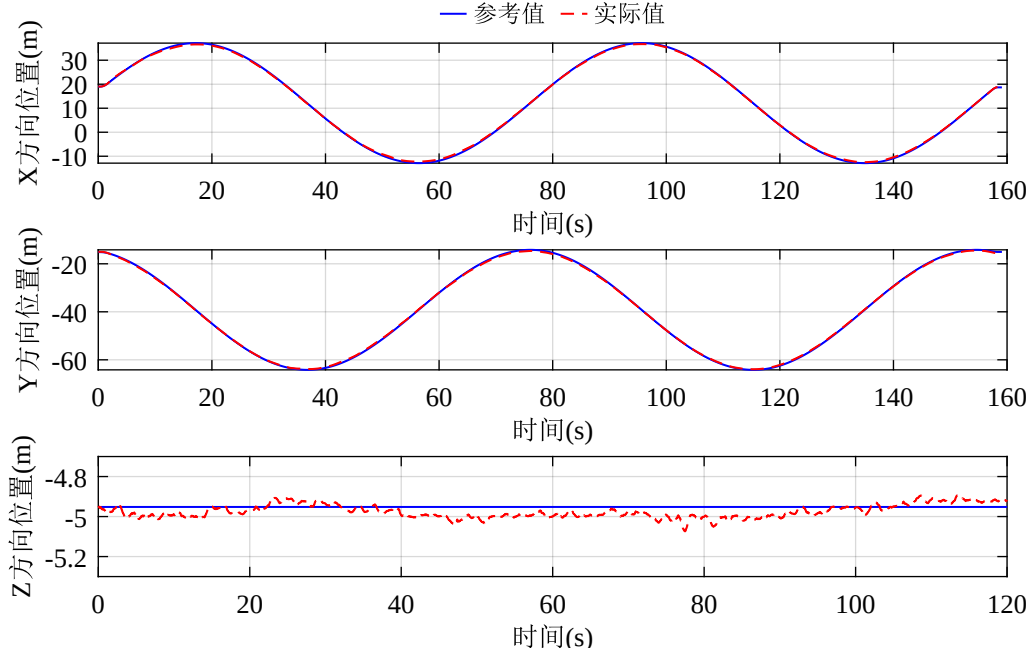


图 2-10 圆弧轨迹飞行实验位置跟踪结果

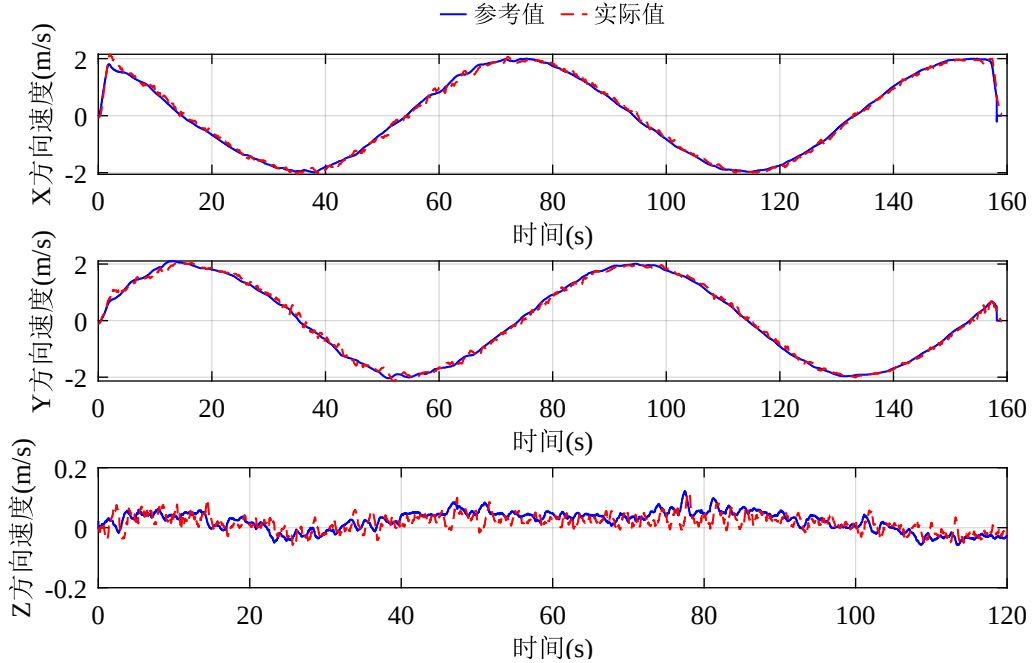


图 2-11 圆弧轨迹飞行实验速度跟踪结果

根据实验结果可知，试验样机在圆弧轨迹跟踪过程中均能够保持在期望轨迹附近，实验过程中三轴位置的最大误差分别为  $[0.68 \ 0.57 \ 0.16]\text{m}$ ，误差情况与点对点轨迹跟

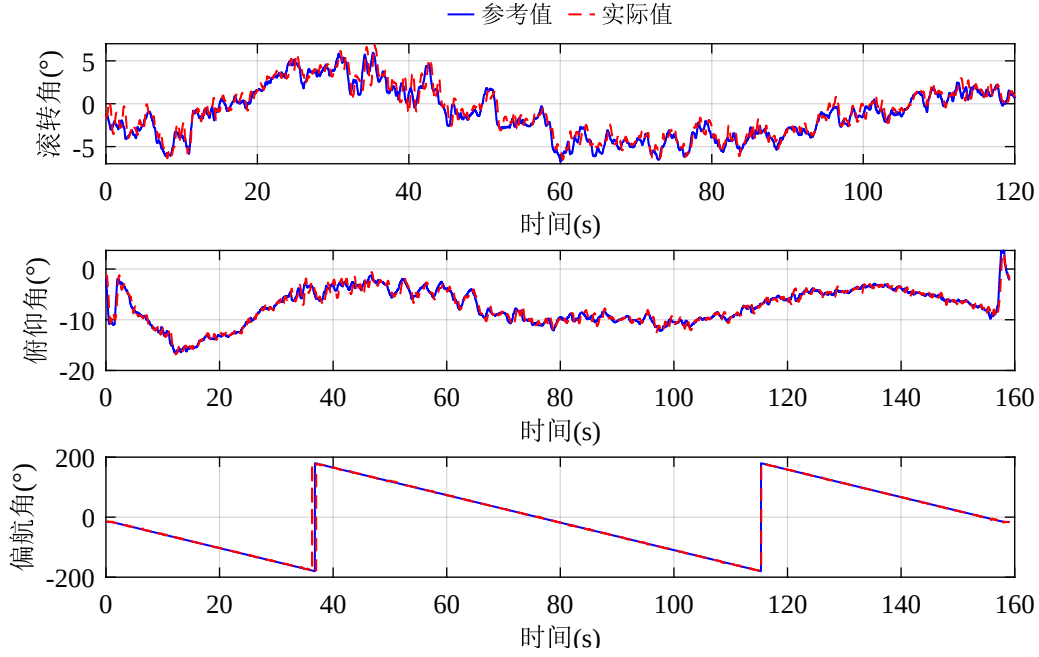


图 2-12 圆弧轨迹飞行实验姿态跟踪结果

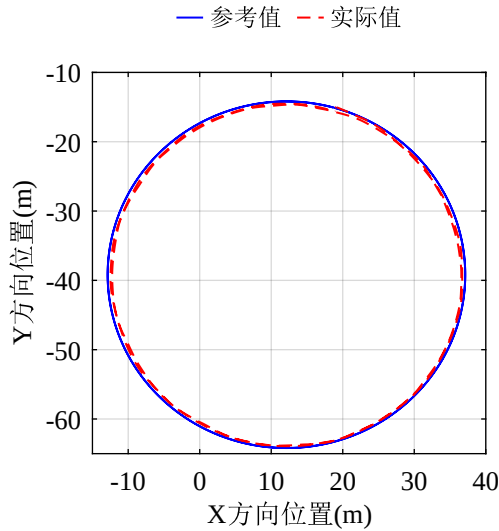


图 2-13 圆弧轨迹飞行实验二维平面轨迹结果

跟踪误差相似，整体误差较小，满足常规任务要求。由于期望轨迹是圆形，所以期望位置投影到 X 方向和 Y 方向上都为正弦曲线，期望速度信号分量包括期望位置变化和位置误差的叠加，所以也近似为正弦曲线。各方向最大速度误差分别为  $[0.26 \ 0.32 \ 0.11]\text{m/s}$ ，均在合理范围内。对于姿态响应曲线，除了俯仰角变化较大外，飞行过程中还需要一定的向心力，导致滚转角也会有适度的变化，偏航角会根据期望轨迹均匀地线性变化，姿态通道最大跟踪误差分别为  $[2.69 \ 4.75 \ 3.55]^\circ$ ，整体跟踪情况良好。

通过二维平面轨迹可以更加直观地看出，试验样机在整个轨迹飞行过程中能够较好地跟踪期望的圆形轨迹，证明了位置控制器对圆弧轨迹同样有较好的跟踪性能。

通过上述两个实际飞行实验，进一步在真实平台上证明了所设计的基于 INDI 的位置控制器具有良好的的轨迹跟踪性能，整个飞行过程中位置误差较小，具备一定的实践意义。

## 2.4 本章小结

本章针对 DFUAV 的位置控制问题展开研究，实现了适用于 DFUAV 的基于 INDI 的位置控制器设计方案。首先对位置动力学模型简化处理，引入具有加速度量纲的位置控制的虚拟控制输入，便于与加速度计测量值统一。然后，基于加速度计测量值可以反映出系统的受力这一特点，拓展了第三章介绍的 INDI 方法，设计了基于 INDI 的位置控制器，形成了外环-内环协同的扰动观测补偿机制。进一步，根据虚拟控制输入计算涵道风扇转速并且通过几何方法解算期望的姿态角，实现了位置-姿态的协同控制。最后通过一系列 MATLAB 仿真与实际飞行实验验证了所设计的基于 INDI 的位置控制器的性能。实验结果表明，所设计的基于 INDI 的位置控制器具有较强的抗风性能和轨迹跟踪性能，能够满足 DFUAV 的位置控制需求。

## 第三章 涵道风扇式无人机的轨迹规划设计

为了提升 DFUAV 的自主性，满足错综复杂环境下的飞行需求，本章针对 DFUAV 的轨迹规划问题展开研究。根据第一章的介绍分析，轨迹规划过程一般分为路径规划和轨迹优化两个阶段。在路径规划阶段，为了在复杂环境中快速找到无碰撞的初始路线，本文选用基于采样的 RRT 全局路径搜索算法。但常规的 RRT 算法存在采样效率低下和搜索路径非最优的问题，因此本章针对上述问题提出 C-RRT\* 算法以加快路径搜索速度，同时在搜索过程中对路径进行优化，保证总路径长度尽可能短。在轨迹优化阶段，针对不平滑且不满足 DFUAV 动力学约束的初始路径，采用 B 样条曲线对其进行平滑处理，以保证 DFUAV 的飞行安全性和稳定性。

本章安排如下：首先介绍传统的 RRT 路径搜索算法及其改进算法 RRT-Connect 算法和 RRT\* 算法的基本原理，然后结合 RRT-Connect 算法和 RRT\* 算法的优势提出 C-RRT\* 算法。接下来介绍 B 样条曲线的基本原理，并使用 B 样条曲线法对 C-RRT\* 算法寻找的初始路径进行优化。最后使用第四章设计的 DFUAV 的基于 INDI 的位置控制器对轨迹规划结果进行跟踪实验。

### 3.1 RRT 算法及其改进算法理论介绍

#### 3.1.1 RRT 算法

快速扩展随机树 (Rapidly-exploring Random Tree, RRT) 算法是一种随机采样的路径搜索算法，其基本思想是从树的根节点 (起点) 出发，通过随机采样的方式不断扩展树的子节点，一旦扩展到目标点附近，那么就可以通过回溯树的路径来生成从起点到目标点的路径。RRT 算法的伪代码如算法3-1所示：

算法中， $T$  表示 RRT 随机树节点的集合， $E$  表示 RRT 随机树的边集合，表示节点之间的连接关系。算法在初始阶段将起点  $x_{start}$  作为树的初始节点加入集合  $T$ ，并将边集合置为空集。然后循环在可行域内随机采样，对每一个采样点  $x_{rand}$ ，都可以找到集合  $T$  中距离  $x_{rand}$  最近的节点  $x_{near}$ 。然后以  $x_{near}$  为起点，在  $x_{near} - x_{rand}$  方向上以固定的步长  $t$  生成一个新节点  $x_{new}$ ，并检查  $x_{new}$  和  $x_{near}$  之间的连线与障碍物是否存在碰撞，如果不存在碰撞，则将  $x_{new}$  加入到集合  $T$  中并且将边  $\{x_{near}, x_{new}\}$  添加到集合  $E$  中。接下来判断新节点  $x_{new}$  是否与目标点  $x_{goal}$  的距离小于设定的阈值  $d_{th}$ ，如果是则跳出循环返回路径搜索结果，否则继续下一次迭代过程，直至到达目标点附近为止。

**Algorithm 3-1 RRT 算法**


---

```

1:  $T \leftarrow x_{start}; E \leftarrow \emptyset;$ 
2: while true do
3:    $x_{rand} \leftarrow \text{RandomSample}();$ 
4:    $x_{near} \leftarrow \text{GetNearest}(x_{rand}, T);$ 
5:    $x_{new} \leftarrow \text{GetNewNode}(x_{rand}, x_{near}, t);$ 
6:   if  $\text{ObstacleFree}(x_{new}, x_{near})$  then
7:      $T \leftarrow T \cup \{x_{new}\}; E \leftarrow E \cup \{x_{near}, x_{new}\};$ 
8:     if  $\text{dist}(x_{new}, x_{goal}) < d_{th}$  then
9:       break;
10:    end if
11:  end if
12: end while
13: return  $(T, E)$ 
    
```

---

最后返回随机树的节点集合和边集合，通过回溯的方式得到从起点到目标点的路径。

根据 RRT 算法的原理可知，如果存在从起点到目标点的无碰撞路径，那么在有限的时间内，RRT 算法一定会找到一条可行的路径。图3-1展示了 RRT 算法仿真实验结果。

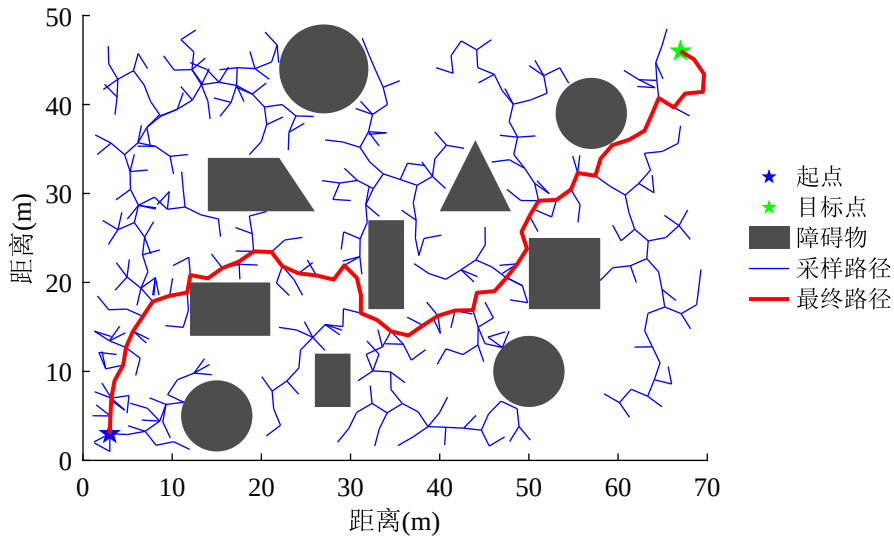


图 3-1 RRT 算法仿真实验结果

传统的 RRT 算法存在采样效率低下和搜索路径非最优的问题：一方面，RRT 算法每次都是在可行域内均匀随机采样，缺乏目标导向，这就会导致在搜索可行路径的过程中可能会搜索到大量的无效节点，从而浪费大量时间和计算资源；另一方面，RRT 算法在搜索路径的时候没有优化过程，导致最终路径可能经过许多不必要的节点，从而导致路径长度较长，非最优路径。针对 RRT 算法的不足，研究者们基于 RRT 算法提出了一些改进的算法，如 RRT-Connect 算法和 RRT\* 算法等。

### 3.1.2 RRT-Connect 算法和 RRT\* 算法

#### 3.1.2.1 RRT-Connect 算法

为了解决传统的 RRT 算法搜索路径的单源性和搜索效率低下的局限性，有研究者提出了 RRT-Connect 算法，该算法相比 RRT 算法使用了双向协同搜索架构和贪婪扩展策略，其基本思想是分别构建以起点和以目标点为根节点的两棵随机树，通过对两棵树交替执行拓展操作逼近彼此，并且引入贪婪扩展策略以加快路径搜索速度。RRT-Connect 算法的伪代码如算法3-2所示：

**Algorithm 3-2** RRT-Connect 算法

---

```

1:  $T_1 \leftarrow x_{start}; E_1 \leftarrow \emptyset;$ 
2:  $T_2 \leftarrow x_{end}; E_2 \leftarrow \emptyset;$ 
3:  $Final \leftarrow false;$ 
4: while not  $Final$  do
5:    $x_{rand} \leftarrow RandomSample();$ 
6:    $x_{near1} \leftarrow GetNearest(x_{rand}, T_1);$ 
7:    $x_{new1} \leftarrow GetNewNode(x_{rand}, x_{near1}, t);$ 
8:   if  $ObstacleFree(x_{new1}, x_{near1})$  then
9:      $T_1 \leftarrow T_1 \cup \{x_{new1}\}; E_1 \leftarrow E_1 \cup \{x_{near1}, x_{new1}\};$ 
10:     $x_{near2} \leftarrow GetNearest(x_{new1}, T_2);$ 
11:     $x_{new2} \leftarrow GetNewNode(x_{new1}, x_{near2}, t);$ 
12:    while  $ObstacleFree(x_{new2}, x_{near2})$  do
13:       $T_2 \leftarrow T_2 \cup \{x_{new2}\}; E_2 \leftarrow E_2 \cup \{x_{near2}, x_{new2}\};$ 
14:       $x_{near\_tmp} \leftarrow GetNearest(x_{new2}, T_1);$ 
15:      if  $ObstacleFree(x_{new2}, x_{near\_tmp})$  and  $dist(x_{new2}, x_{near\_tmp}) < d_{th}$  then
16:         $Final \leftarrow true;$ 
17:        break;
18:      end if
19:       $x_{near2} \leftarrow x_{new2};$ 
20:       $x_{new2} \leftarrow GetNewNode(x_{new1}, x_{near2}, t);$ 
21:    end while
22:     $swap(T_1, T_2); swap(E_1, E_2);$ 
23:  end if
24: end while
25: return  $(T_1, T_2, E_1, E_2)$ 

```

---

根据算法流程可知，相比于 RRT 算法，RRT-Connect 算法在初始化阶段建立了两棵随机树，并将起点和目标点分别作为两棵随机树的初始节点，然后执行与 RRT 算法相同的步骤来循环地随机寻找新节点  $x_{new1}$ 。在随机找到与障碍物无碰撞的新节点后，

根据新节点的位置可以找到随机树  $T_2$  上与新节点距离最近的节点  $x_{near2}$ 。接下来执行贪婪扩展策略，以  $x_{near2}$  为起点，在  $x_{near2} - x_{new1}$  方向上以固定的步长  $t$  生成新节点  $x_{new2}$ ，并检查  $x_{new2}$  与  $x_{near2}$  之间的连线是否与障碍物存在碰撞，如果不存在碰撞，则将  $x_{new2}$  加入集合  $T_2$  中并且将边  $\{x_{near2}, x_{new2}\}$  添加到集合  $E_2$  中。然后从  $T_1$  上找到距离  $x_{new2}$  的最近节点  $x_{near\_tmp}$ ，判断  $x_{new2}$  与  $x_{near\_tmp}$  之间是否与障碍物无碰撞以及两节点之间的距离是否小于  $d_{th}$ ，如果两个条件都满足的话，则跳出循环并返回路径搜索结果。如果上述任一条件不满足，则继续执行贪婪策略， $T_2$  以  $x_{new2}$  为新的起点 (为方便编程表示，此处将其设为  $x_{near2}$ ) 继续向  $x_{new1}$  方向生成新节点，直到检测到新节点与  $x_{near2}$  之间存在障碍物为止。如果贪婪扩展过程中检测到障碍物碰撞，则交换两棵随机树的节点集合和边集合，继续随机采样过程，直至两棵树存在叶子节点之间的距离小于  $d_{th}$  且无障碍物。最后根据两棵树的节点集合和边集合回溯得到路径搜索的结果。

图3-2展示了使用 RRT-Connect 算法进行路径规划仿真的结果。由于 RRT-Connect 算法中的两棵树是双向生长的，并且通过贪婪策略优先在两棵树之间节点的可扩展区域进行生长，使得两棵树可以快速相交。所以 RRT-Connect 相比于 RRT 算法大量减少了无效节点的生成，加快了路径搜索的速度。但是由于 RRT-Connect 算法的单次查询性质，其仍然存在路径非最优的问题。

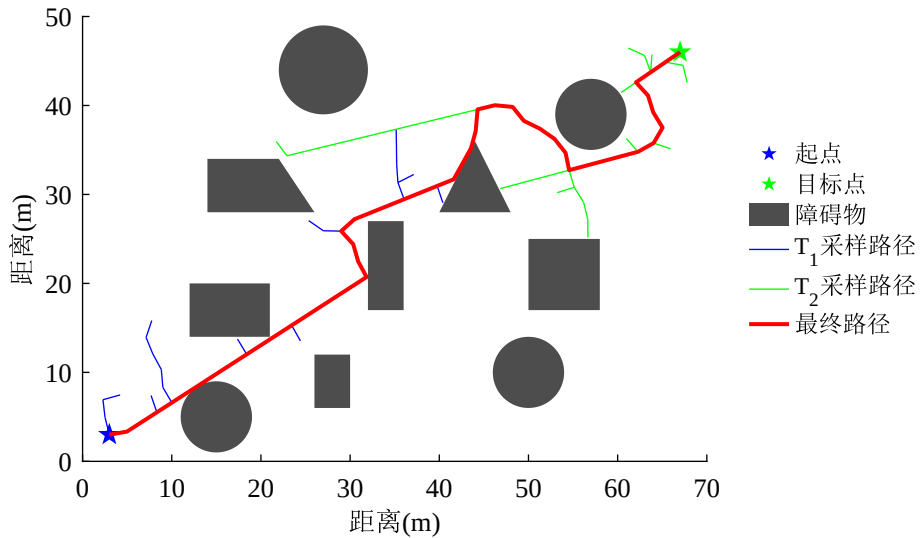


图 3-2 RRT-Connect 算法仿真实验结果

### 3.1.2.2 RRT\* 算法

为了解决 RRT 算法中路径非最优的问题，有研究学者提出了 RRT\* 算法。RRT\* 算法属于渐进优化的算法，其基本思想是在随机树中添加新节点时，通过对新节点邻域半

径内的节点进行选择性地重连操作，以保证选取的是当前范围内的最优父节点，并借助新节点使用重布线优化机制动态优化树结构，进一步确保路径的渐进最优性。RRT\* 算法的伪代码如算法3-3所示。

---

**Algorithm 3-3 RRT\* 算法**


---

```

1:  $T \leftarrow x_{start}; E \leftarrow \emptyset; \text{Cost}(x_{start}) \leftarrow 0;$ 
2: while true do
3:    $x_{rand} \leftarrow \text{RandomSample}();$ 
4:    $x_{near} \leftarrow \text{GetNearest}(x_{rand}, T);$ 
5:    $x_{new} \leftarrow \text{GetNewNode}(x_{rand}, x_{near}, t);$ 
6:   if  $\text{ObstacleFree}(x_{new}, x_{near})$  then
7:      $\text{Cost}(x_{new}) \leftarrow \text{Cost}(x_{near}) + t;$ 
8:      $T \leftarrow T \cup \{x_{new}\}; E \leftarrow E \cup \{x_{near}, x_{new}\};$ 
9:      $C \leftarrow \text{GetNodeInCircle}(x_{new}, T, r);$ 
10:    if  $\text{IsEmpty}(C)$  then
11:       $x_{near\_new} \leftarrow \text{GetLowestCost}(x_{new}, C);$ 
12:       $E \leftarrow E \setminus \{x_{near}, x_{new}\}; E \leftarrow E \cup \{x_{near\_new}, x_{new}\};$ 
13:      for  $x_i \in C$  do
14:        if  $\text{dist}(x_i, x_{new}) + \text{Cost}(x_{new}) < \text{Cost}(x_i)$  then
15:           $x_{parent} \leftarrow \text{Parent}(x_i);$ 
16:           $E \leftarrow E \setminus \{x_i, x_{parent}\}; E \leftarrow E \cup \{x_i, x_{new}\};$ 
17:        end if
18:      end for
19:    end if
20:    if  $\text{dist}(x_{new}, x_{goal}) < d_{th}$  then
21:      break;
22:    end if
23:  end if
24: end while
25: return  $(T, E)$ 

```

---

根据 RRT\* 算法流程可知，RRT\* 算法对于每一个节点，都会计算其代价  $\text{Cost}(x_i)$ ，具体值常用距离来表示。对于每一个通过随机采样得到的新节点  $x_{new}$ ，RRT\* 算法会寻找以  $x_{new}$  为中心，半径为  $r$  的邻域内的所有连接无碰撞的节点（节点集合使用  $C$  表示）。如果集合  $C$  不为空集，那么就从  $C$  中选择一个节点  $x_{near\_new}$ ，该节点需满足从  $x_{start}$  到  $x_{near\_new}$  的代价加上从  $x_{near\_new}$  到  $x_{new}$  的代价最小。然后将  $x_{near}$  作为  $x_{new}$  的父节点，同时更新边集合  $E$ ，将原来的边  $\{x_{near}, x_{new}\}$  替换为  $\{x_{near\_new}, x_{new}\}$ 。这一步即为最优父节点选择操作。接下来需要重布线优化树结构，通过遍历  $C$  中的所有节点  $x_i$ ，如果新节点  $x_{new}$  到节点  $x_i$  的距离加上节点  $x_{new}$  的代价小于节点  $x_i$  的代价，

则更新节点  $x_i$  的父节点为新节点  $x_{new}$ 。最后判断新节点  $x_{new}$  是否与目标点  $x_{goal}$  的距离小于设定的距离阈值  $d_{th}$ ，如果是则跳出循环返回路径搜索结果，否则继续下一次迭代过程，直至找到一条可行路径。

RRT\* 算法的仿真结果如图3-3所示。可以看出，RRT\* 算法相比 RRT 算法和 RRT-Connect 算法，搜索的路径的长度有明显的减小。这得益于 RRT\* 算法的渐进优化机制，在搜索过程中动态地对树结构进行优化，使路径向全局最优靠拢。Karaman 等学者证明了随着搜索节点数目的增加，RRT\* 算法得到的路径会逐渐收敛到全局最优路径<sup>[8]</sup>。

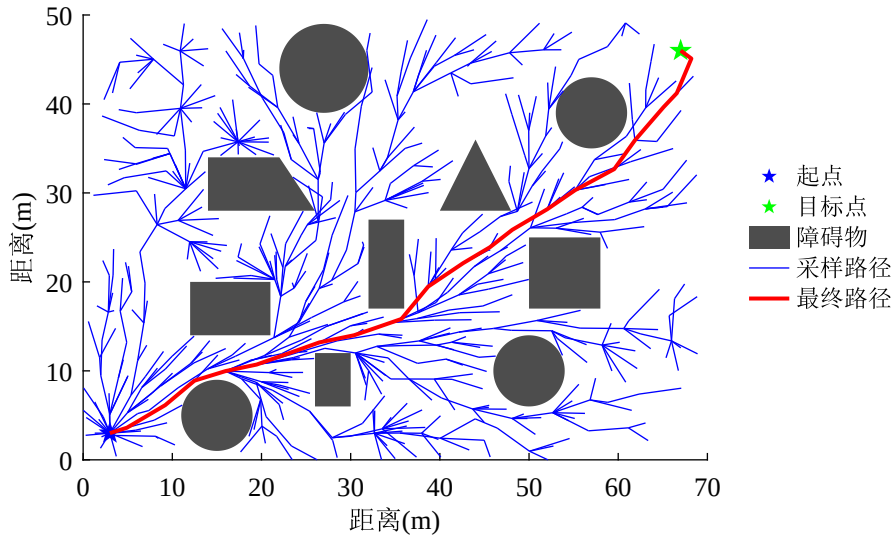


图 3-3 RRT\* 算法仿真实验结果

但也正因为 RRT\* 算法的渐进优化机制，对每一个采样点都需要进行最优父节点选择和树结构优化策略，使得 RRT\* 算法相比于 RRT 算法多了许多代价比较和碰撞检测的过程。进而导致 RRT\* 算法在搜索过程中的计算量很大，耗时很长。在搜索空间大、空间维数高的情况下，RRT\* 算法的搜索效率会受到很大的影响。

### 3.2 C-RRT\* 算法

本节针对 RRT 算法的采样效率低和搜索路径非最优的问题，结合 RRT-Connect 算法和 RRT\* 算法各自的优势，提出了 C-RRT\* 算法。一方面，C-RRT\* 算法采用 RRT-Connect 算法的双向搜索架构和贪婪扩展策略，有侧重地优先向目标点方向 (或起点方向) 搜索，大量地减少无效节点的生成，加快路径搜索的速度；另一方面，C-RRT\* 算法引入 RRT\* 算法的渐进优化机制，对每一个新节点  $x_{new}$  都执行最优父节点选择和周围邻域半径内的树结构优化策略，显著降低路径的累计代价。

### 3.2.1 冗余节点删除策略

经过采样搜索的路径存在转折角度小，弯曲多的特征，不利于 DFUAV 的飞行安全。因此，C-RRT\* 算法引入了冗余节点删除策略，遍历搜索得到的初始路径，在与障碍物无碰撞的情况下删除路径上的冗余节点。冗余节点删除策略的伪代码如算法3-4所示：

**Algorithm 3-4** RemoveRedundantNodes

```

1: Input: initPath
2: optiPath  $\leftarrow$  initPath;
3: index  $\leftarrow$  1;
4: while index  $\leq$  optiPath.size()-2 do
5:   if ObstacleFree(optiPath[index], optiPath[index+2]) then
6:     optiPath  $\leftarrow$  optiPath.remove(index+1);
7:   else
8:     index  $\leftarrow$  index + 1;
9:   end if
10: end while
11: Output: optiPath

```

根据算法流程，冗余节点删除策略会遍历搜索得到的初始路径，对路径上除最后两个节点外的每一个节点 optiPath[index]，都会检查节点 optiPath[index] 和节点 optiPath[index+2] 之间的连线是否与障碍物存在碰撞，如果无碰撞，则可以删除节点 optiPath[index+1]，直至遍历完整个路径。最后返回优化后的路径。以图3-1中经 RRT 算法搜索得到的路径为例进行测试，测试结果如图3-4所示。经过对比可以发现，使用冗余节点删除策略优化后的路径减少了许多弯折，初始路径长度明显缩短。

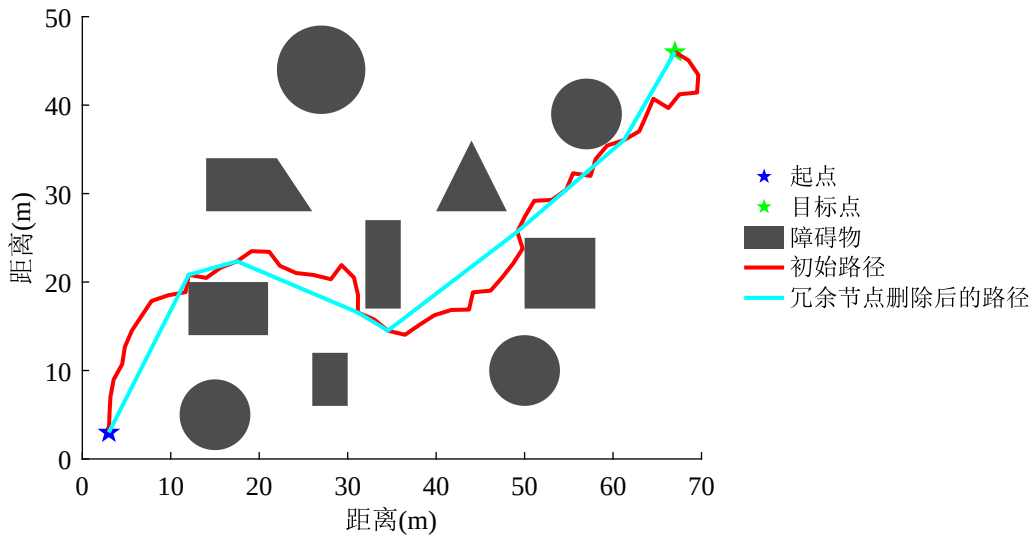


图 3-4 冗余节点删除前后仿真实验对比结果

### 3.2.2 C-RRT\* 算法流程

基于 5.1 节介绍的 RRT-Connect 算法和 RRT\* 算法的基本原理，并且结合冗余节点删除策略，可以得到 C-RRT\* 算法的伪代码如算法3-5所示：

**Algorithm 3-5** C-RRT\* 算法

---

```

1:  $T_1 \leftarrow x_{start}; E_1 \leftarrow \emptyset; \text{Cost}(x_{start}) \leftarrow 0;$ 
2:  $T_2 \leftarrow x_{end}; E_2 \leftarrow \emptyset; \text{Cost}(x_{end}) \leftarrow 0;$ 
3:  $\text{Final} \leftarrow false;$ 
4: while not  $\text{Final}$  do
5:    $x_{rand} \leftarrow \text{RandomSample}();$ 
6:    $x_{near1} \leftarrow \text{GetNearest}(x_{rand}, T_1);$ 
7:    $x_{new1} \leftarrow \text{GetNewNode}(x_{rand}, x_{near1}, t);$ 
8:   if  $\text{ObstacleFree}(x_{new1}, x_{near1})$  then
9:      $\text{Cost}(x_{new1}) \leftarrow \text{Cost}(x_{near1}) + t;$ 
10:     $T_1 \leftarrow T_1 \cup \{x_{new1}\}; E_1 \leftarrow E_1 \cup \{x_{near1}, x_{new1}\};$ 
11:     $C_1 \leftarrow \text{GetNodeInCircle}(x_{new1}, T_1, r);$ 
12:    if  $\text{IsEmpty}(C_1)$  then
13:       $E_1 \leftarrow \text{RrtStarAlgorithm}(x_{new1}, C_1, E_1);$ 
14:    end if
15:     $x_{near2} \leftarrow \text{GetNearest}(x_{new1}, T_2);$ 
16:     $x_{new2} \leftarrow \text{GetNewNode}(x_{new1}, x_{near2}, t);$ 
17:    while  $\text{ObstacleFree}(x_{new2}, x_{near2})$  do
18:       $\text{Cost}(x_{new2}) \leftarrow \text{Cost}(x_{near2}) + t;$ 
19:       $T_2 \leftarrow T_2 \cup \{x_{new2}\}; E_2 \leftarrow E_2 \cup \{x_{near2}, x_{new2}\};$ 
20:       $C_2 \leftarrow \text{GetNodeInCircle}(x_{new2}, T_2, r);$ 
21:      if  $\text{IsEmpty}(C_2)$  then
22:         $E_2 \leftarrow \text{RrtStarAlgorithm}(x_{new2}, C_2, E_2);$ 
23:      end if
24:       $x_{near\_tmp} \leftarrow \text{GetNearest}(x_{new2}, T_1);$ 
25:      if  $\text{ObstacleFree}(x_{new2}, x_{near\_tmp})$  and  $\text{dist}(x_{new2}, x_{near\_tmp}) < d_{th}$  then
26:         $\text{Final} \leftarrow true;$ 
27:        break;
28:      end if
29:       $x_{near2} \leftarrow x_{new2};$ 
30:       $x_{new2} \leftarrow \text{GetNewNode}(x_{new1}, x_{near2}, t);$ 
31:    end while
32:     $\text{swap}(T_1, T_2); \text{swap}(E_1, E_2);$ 
33:  end if
34: end while
35:  $(T_1, T_2) \leftarrow \text{RemoveRedundantNodes}(T_1, T_2);$ 
36: return  $(T_1, T_2, E_1, E_2)$ 

```

---

C-RRT\* 算法流程中体现出了 RRT-Connect 算法和 RRT\* 算法各自的特点：既包含了 RRT-Connect 算法的使用两棵树双向搜索架构和贪婪扩展策略，减少无效节点的生成，加快了路径搜索速度；又引入了 RRT\* 算法的最优父节点选择和重布线优化策略，使得 C-RRT\* 算法同样有渐进优化的特性。为了减少伪代码的冗余显示，算法3-5中简写了 RRT\* 算法相关的伪代码，使用函数 `RrtStarAlgorithm` 封装了 RRT\* 算法伪代码3-3中的第 12 至第 19 行的优化操作。对于寻得的初始路径，使用 `RemoveRedundantNodes` 函数对路径上的冗余节点进行删除，得到最终的路径搜索结果。

图3-5展示了 C-RRT\* 算法的仿真实验结果。可以看出，C-RRT\* 算法搜索的节点数目远小于 RRT 算法和 RRT\* 算法，极大地减少了路径规划的时间；并且搜索的最终路径长度相比 RRT 算法和 RRT-Connect 算法明显缩短，在路径的弯曲度上也有明显的改善。

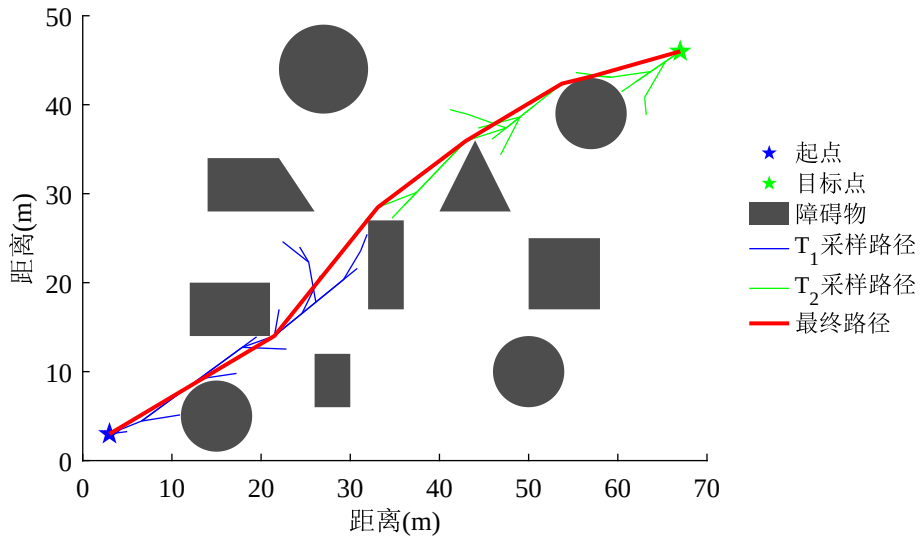


图 3-5 C-RRT\* 算法仿真实验结果

### 3.3 基于 B 样条曲线的轨迹优化

基于采样的路径规划算法得到的可行路径通常是由一系列离散节点组成，这些节点之间的连线构成了路径的折线形状，已具备避障能力。然而，折线路径的特点是路径的转向角度突变，在转折点处的路径曲率不连续，进而导致运动控制指令(如速度、加速度)出现阶跃变化。如果直接跟踪折线路径，将导致执行机构的高频震荡，降低轨迹跟踪精度。因此，需要对采样得到的初始路径进行平滑处理，使得路径曲率连续。参数曲线方程得到的曲线因严格的数学连续性保证，一般都会满足高阶导数连续。其中，B 样条曲线是参数曲线中具有代表性的一种，其凭借其局部支撑性和凸包性等优势，在轨迹优化领域得到广泛应用。文献[9-10]都是使用采样方法得到无碰撞初始路径，然后使用

B 样条曲线对路径进行平滑优化。本节将介绍 B 样条曲线的相关理论，并使用 B 样条曲线对 C-RRT\* 算法搜索的初始路径进行轨迹优化。

### 3.3.1 B 样条曲线理论

B 样条 (B-spline) 曲线是一种常用的样条曲线，样条曲线的特点是由一组控制点进行参数化定义。设一共有  $n + 1$  个控制点  $P_0, P_1, \dots, P_n$ ，这些控制点用于定义样条曲线的形状。B 样条曲线的定义如下：

$$P(t) = \begin{bmatrix} P_0 & P_1 & \dots & P_n \end{bmatrix} \begin{bmatrix} B_{0,k}(t) \\ B_{1,k}(t) \\ \vdots \\ B_{n,k}(t) \end{bmatrix} = \sum_{i=0}^n B_{i,k}(t) P_i \quad (3-1)$$

式中，每一个控制点都对应一个 B 样条基函数， $B_{i,k}(t)$  表示第  $i$  个  $k$  阶 B 样条基函数，其中阶数  $k \geq 1$ 。B 样条基函数的递归定义如下：

$$\begin{cases} B_{i,1}(t) = \begin{cases} 1 & t_i \leq t < t_{i+1} \\ 0 & \text{其他} \end{cases} \\ B_{i,k}(t) = \frac{t-t_i}{t_{i+k-1}-t_i} B_{i,k-1}(t) + \frac{t_{i+k}-t}{t_{i+k}-t_{i+1}} B_{i+1,k-1}(t) & t_i \leq t < t_{i+k} \end{cases} \quad (3-2)$$

式中， $(t_0, t_1, \dots, t_{n+k})$  是一组非递减序列，称为节点矢量。B 样条基函数的定义中可能出现  $\frac{0}{0}$  的情况，此时规定  $\frac{0}{0} = 0$ 。

根据 B 样条基函数的定义可知，当阶数  $k \geq 2$  时，第  $i$  个  $k$  阶 B 样条基函数由第  $i$  个  $k-1$  阶 B 样条基函数和第  $i+1$  个  $k-1$  阶 B 样条基函数通过  $t$  线性组合得到。根据该递推规律，可以用图 3-6 直观地表示 B 样条基函数的递推关系。

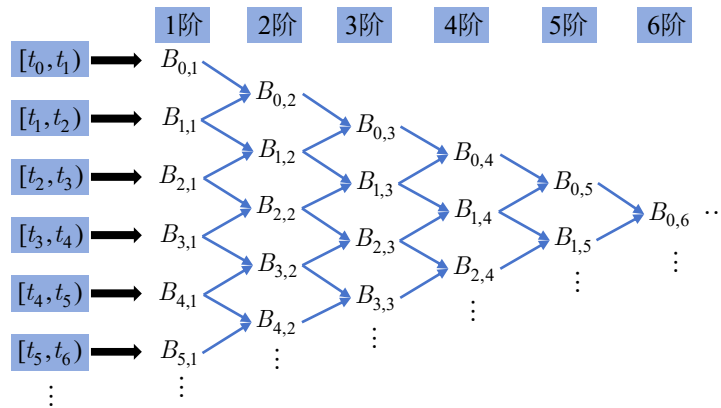


图 3-6 B 样条基函数递推关系示意图

## 3.4 仿真实验分析

### 3.4.1 C-RRT\* 算法综合性能验证与对比分析

需要特别指出的是, 尽管传统的 RRT 算法及其改进型算法 (如 RRT-Connect、RRT\*) 在路径规划领域已展现出广泛的适用性, 但其基于随机采样的本质特性仍会导致规划结果的不确定性。为系统性评估本文提出的 C-RRT\* 算法的综合性能, 本文构建了仿真测试环境, 并基于蒙特卡洛方法对四类算法 RRT、RRT-Connect、RRT\* 和 C-RRT\* 进行各 500 次独立重复实验。通过对实验结果统计分析, 从平均路径搜索时间、平均路径搜索节点数目、平均路径长度和平均路径弯曲度四个维度量化算法的性能差异。仿真实验结果如表3-1所示。

表 3-1 C-RRT\* 算法性能评估结果对比分析

| 算法名称        | 平均搜索时间 (s) | 平均节点数目 | 平均路径长度 (m) | 平均路径弯曲度 (rad) |
|-------------|------------|--------|------------|---------------|
| RRT         | 0.29       | 525.00 | 102.18     | 32.04         |
| RRT-Connect | 0.044      | 70.52  | 93.25      | 14.21         |
| RRT*        | 30.44      | 521.50 | 84.88      | 7.74          |
| C-RRT*      | 0.26       | 69.05  | 85.91      | 2.32          |

下面根据表格数据分析四类算法的性能差异。

平均路径搜索时间反映了路径搜索算法的实时性, 这在资源受限的嵌入式系统中尤为重要。RRT-Connect 算法的平均搜索时间最短, 仅为 0.044s, 这得益于 RRT-Connect 算法的双向搜索和贪婪策略。RRT 算法和 C-RRT\* 算法的平均搜索时间差不多, 分别为 0.29s 和 0.26s, 但它们耗时在不同的方面。RRT 算法主要耗时在采样过程中, 而 C-RRT\* 算法主要耗时在使用 RRT\* 算法的优化过程中。RRT\* 算法的平均搜索时间为 30.44s, 相比其他算法要慢许多, 一方面是因为 RRT 算法采样点多, 另一方面是因为 RRT 算法的优化过程导致计算量很大, 耗时很长。

平均搜索的节点数目反映了路径搜索算法的空间复杂度, 是内存占用的直接体现。使用双向搜索和贪婪策略的 RRT-Connect 算法和 C-RRT\* 算法搜索的平均节点数目较少, 分别为 70.52 个和 69.05 个。而在整个可行域内均匀随机采样的 RRT 算法和 RRT\* 算法的平均节点数目较多, 分别为 525.00 个和 521.50 个。

平均路径长度直接反映了路径的经济性, 路径长度越短, 通常能量消耗越低。RRT

算法搜索得到的平均路径长度最长，达到 102.18m。一方面是因为 RRT 算法在搜索路径的时候没有目标导向，搜索过程完全随机，另一方面是因为 RRT 算法没有优化过程，导致最终路径长度较长。其次是 RRT-Connect 算法搜索的平均路径长度为 93.25m，相比于 RRT 算法多了双向搜索和贪婪策略，路径长度有所减小。RRT\* 算法和 C-RRT\* 算法由于引入了渐进优化机制，降低路径的累计代价，所以平均路径长度相对最短，分别为 84.88m 和 85.91m。

此处的平均路径弯曲度定义为搜索的路径的转向角度，该值反映了路径的平滑程度。更平滑的路径更容易被轨迹跟踪控制器执行，减小机械结构磨损。RRT 算法的平均路径弯曲度最大，达到 32.04rad，远远高于其他三类算法。RRT-Connect 算法的平均路径弯曲度为 14.21rad，相比于 RRT 算法有所减小，这是因为 RRT-Connect 算法的贪婪策略使得搜索的路径在障碍物较少的区域都为直线路径，这一点从图3-2中可以明显看出。其次是 RRT\* 算法的平均路径弯曲度为 7.74rad，这得益于 RRT\* 渐进优化机制，对路径进行了优化，使得路径更加平滑。平均路径弯曲度最小的是 C-RRT\* 算法，为 2.32rad，这是因为 C-RRT\* 算法不仅综合了 RRT-Connect 算法和 RRT\* 算法的优势，还使用冗余节点删除策略，进一步平滑路径。

根据以上对比分析可得，本文提出的 C-RRT\* 算法在比较的四个维度上综合性能表现最好。

### 3.5 本章小结

## 参考文献

- [1] Wang X, van Kampen E J, Chu Q, et al. Stability Analysis for Incremental Nonlinear Dynamic Inversion Control[J]. Journal of Guidance, Control, and Dynamics, 2019, 42(5): 1116-1129.
- [2] Pfeifle O, Fichter W. Energy Optimal Control Allocation for INDI Controlled Transition Aircraft[C]//. 2021.
- [3] Härkegård O, Glad S T. Resolving Actuator Redundancy—Optimal Control vs. Control Allocation[J]. Automatica, 2005, 41(1): 137-144.
- [4] 蒙超恒, 裴海龙, 程子欢. 涵道风扇式无人机的优先级控制分配[J]. 航空学报, 2020, 41(10): 327-338.
- [5] Gamagedara K, Lee T. Geometric Adaptive Controls of a Quadrotor Unmanned Aerial Vehicle with Decoupled Attitude Dynamics[J]. Journal of Dynamic Systems, Measurement, and Control, 2021, 144(3): 031002. eprint: [https://asmedigitalcollection.asme.org/dynamicsystems/article-pdf/144/3/031002/6803181/ds\\_144\\_03\\_031002.pdf](https://asmedigitalcollection.asme.org/dynamicsystems/article-pdf/144/3/031002/6803181/ds_144_03_031002.pdf).
- [6] Smeur E J J, Chu Q, Croon G C H E de. Adaptive Incremental Nonlinear Dynamic Inversion for Attitude Control of Micro Air Vehicles[J]. Journal of Guidance, Control, and Dynamics, 2015.
- [7] Smeur E J J, de Croon G C H E, Chu Q. Cascaded Incremental Nonlinear Dynamic Inversion for MAV Disturbance Rejection[J]. Control Engineering Practice, 2018, 73: 79-90.
- [8] Karaman S, Frazzoli E. Sampling-based algorithms for optimal motion planning[J]. The international journal of robotics research, 2011, 30(7): 846-894.
- [9] Wang L, Zhang Y, Guo C. Path Planning for a Prostate Intervention Robot Based on an Improved Bi-RRT Algorithm[J]. IEEE/ASME Transactions on Mechatronics, 2025, 30(1): 668-678.
- [10] Li N, Han S I. Adaptive Bi-Directional RRT Algorithm for Three-Dimensional Path Planning of Unmanned Aerial Vehicles in Complex Environments[J]. IEEE Access, 2025, 13: 23748-23767.